

Data Mining Notes

Academic Year 2026/2027

Contents

1	Introduction to Data Mining and the KDD Process	1
1.1	Definition and Fundamental Motivations	1
1.2	Types of Data Origin: Primary and Secondary	3
1.3	The KDD Process (Knowledge Discovery in Databases)	3
1.4	Taxonomy of Data Mining Tasks	5
1.5	Methodological and Computational Challenges in Data Mining	10
2	Data Understanding and Data Exploration	11
2.1	Objectives of Data Understanding	11
2.2	Types of Datasets	12
2.3	Objects and Attributes	13
2.4	Data Quality	15
2.5	Descriptive Statistics and Univariate Exploration	16
2.6	Multivariate Visualization Techniques	23
2.7	Linear Correlation and Association Analysis	28
2.8	Handling Outliers and Missing Values	36
2.9	Operational Checklist for Data Understanding	39
3	Data Preparation and Data Cleaning	41
3.1	Objectives of Data Preparation	41
3.2	Data Aggregation	43
3.3	Data Reduction via Sampling	44
3.4	Dimensionality Reduction	46
3.5	Feature Subset Selection	48
3.6	Feature Creation	49
3.7	Principal Component Analysis (PCA)	50
3.8	Missing Value and Anomaly Cleaning	54
3.9	Discretization and Binarization	57
3.10	Attribute Transformation and Variance Stabilization	62

4	Data Similarity and Introduction to Cluster Analysis	68
4.1	Proximity: Similarity and Dissimilarity	68
4.2	Euclidean Distance and Metric Properties	69
4.3	Minkowski Distance: A Family of Metrics	70
4.4	Similarity Between Binary Data	71
4.5	Cosine Similarity	72
4.6	Correlation	73
4.7	Information-Based Measures: Entropy and Mutual Information	73
4.8	What is Cluster Analysis?	75
4.9	Ambiguity and Subjectivity of Clusters	76
4.10	Taxonomy of Clusterings	76
4.11	Types of Clusters	77
4.12	Input Data Characteristics and Clustering Algorithms	79

Chapter 1

Introduction to Data Mining and the KDD Process

1.1 Definition and Fundamental Motivations

The constant technological progress in data generation and storage has led to an exponential growth of available information, both in commercial and scientific contexts. The modern paradigm of data collection is often summarized in the principle of gathering any possible data, anywhere and anytime, in the expectation that such data may acquire value not only for the original purpose for which they were collected, but also for secondary purposes not initially foreseen.

Definition 1.1.1 (Data Mining). **Data Mining** is the application of efficient techniques and algorithms to the analysis of large data collections, aimed at the extraction of valid, novel, potentially useful, and interpretable patterns (hidden knowledge).

Data Mining does not consist in the simple execution of queries on traditional databases (for example, retrieving the average balance via SQL), but rather in the automatic or semi-automatic discovery of non-trivial structures, dependencies, or regularities in the data.

1.1.1 Application Perspectives

The motivations behind the adoption of Data Mining techniques can be distinguished primarily according to two points of view:

- **Commercial point of view:** Companies manage enormous volumes of data (*data warehousing*): web data (Yahoo, Facebook), e-commerce

transactions and retail sales (Amazon), credit card records and bank accounts. In the face of increasingly cheap and accessible computing power, competitive pressure requires extracting value from such masses of data to improve and personalize the services offered (for example in *Customer Relationship Management* - CRM).

- **Scientific and social point of view:** In disciplines such as astronomy (*Sky Survey Data*), Earth observation (NASA EOSDIS satellites collect petabytes of environmental data per year), genomics (gene expression, proteomics), neuroimaging (fMRI), and complex numerical simulations, data are generated at such speeds as to make manual analysis impossible. Data Mining supports scientists in automatic analysis and in the guided formulation of new hypotheses.

Big Data represent to all intents and purposes a proxy of social life (purchasing behaviors, lifestyles, opinions, relationship networks, territorial mobility), offering great opportunities to address global issues: from the efficiency of healthcare to the search for clean energy sources, from forecasting climate impacts to optimizing agricultural production.

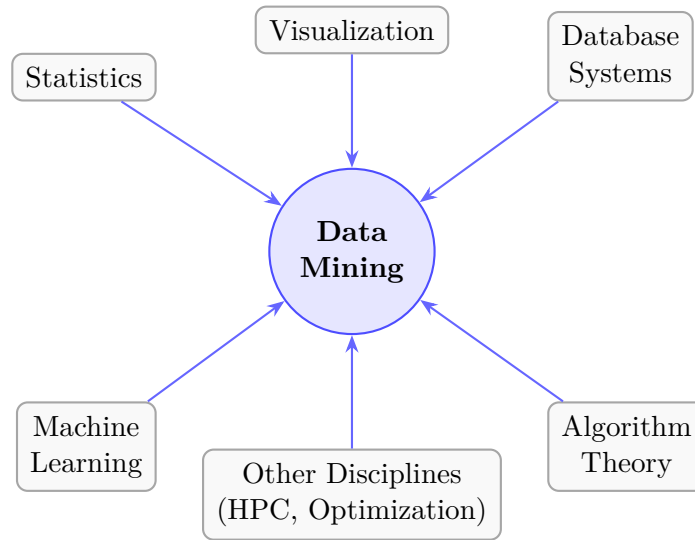
In this modern scenario, applied Artificial Intelligence rests on the foundational paradigm:

$$\mathbf{AI} = \mathbf{Machine\ Learning} + \mathbf{Big\ Data} \quad (1.1)$$

where the availability of massive databases (*Big Data*) provides the empirical substrate and source of information, while *Machine Learning* and *Data Mining* algorithms act as the logical-inductive engine capable of extracting non-trivial generalizations and decision patterns.

1.1.2 A Confluence of Disciplines

Data Mining is not an isolated discipline, but rather a synergistic confluence of several areas of computer science and applied mathematics:



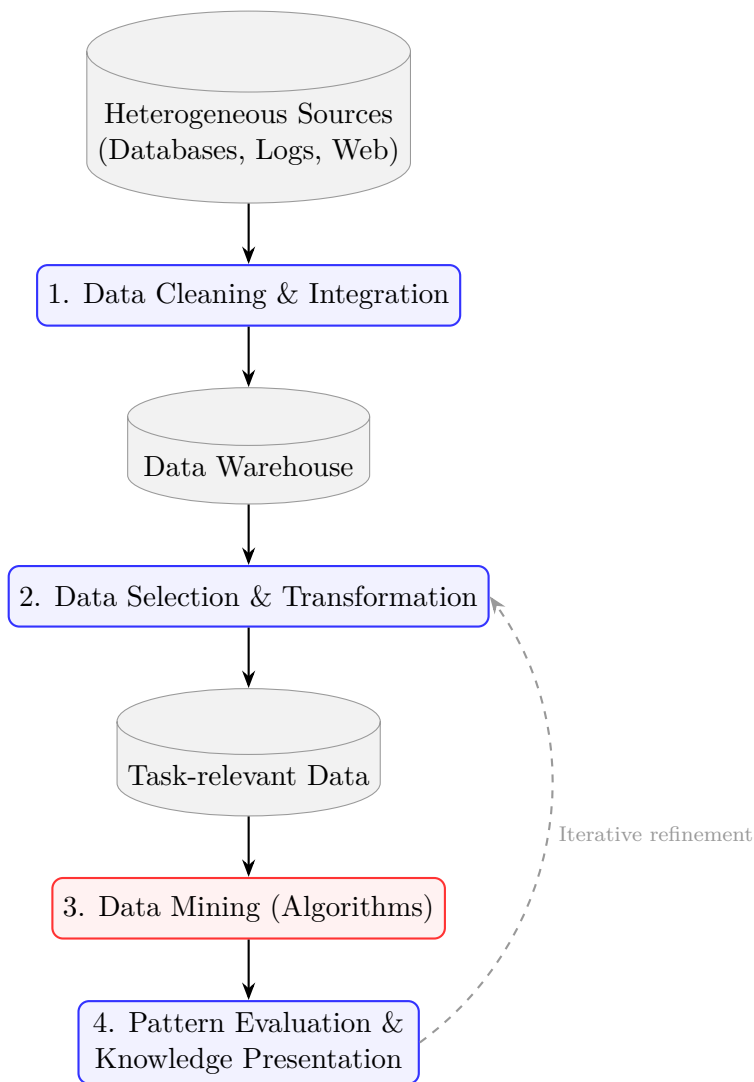
1.2 Types of Data Origin: Primary and Secondary

In setting up an analytical project, it is essential to distinguish the origin of the analyzed data:

- **Primary Data:** Original data collected specifically for a precise investigation objective. They have not undergone any preventive alterations or manipulations by third parties.
- **Secondary Data:** Data that had already been previously collected for operational purposes or different from the current one and made available to the analyst. They often derive from the merging of heterogeneous sources and require deep normalization and quality verification operations.

1.3 The KDD Process (Knowledge Discovery in Databases)

Data Mining constitutes a specific, although central, phase within the broader knowledge discovery process, known as **KDD** (*Knowledge Discovery in Databases*). The KDD is an iterative and interactive process composed of several sequential phases with feedback loops:



Each phase performs specific tasks:

1. **Business Understanding:** Understanding the domain objectives and translating the business problem into a formal data analysis problem.
2. **Data Cleaning and Data Integration:**
 - *Data Cleaning:* removal of noise, correction of syntactic errors, and handling of inconsistencies.
 - *Data Integration:* aggregation and reconciliation of data from multiple and heterogeneous sources, typically flowing into a **Data Warehouse** (a structured and optimized repository for analysis and decision support).

3. Data Selection and Data Transformation:

- *Data Selection*: extraction of the subset of attributes and records strictly relevant to the investigated task (*task-relevant data*).
- *Data Transformation*: transformation of features through aggregations, rescalings, normalizations, or discretizations so that they are suitable for the algorithm.

4. **Data Mining**: The actual execution of intelligent algorithms for the extraction of patterns and mathematical models.

5. **Pattern Evaluation and Knowledge Presentation**: Identification of truly significant patterns through objective and subjective metrics, elimination of redundancy, and graphical visualization of results using tools interpretable by the end user.

1.4 Taxonomy of Data Mining Tasks

Following the classification of Fayyad et al. [1], Data Mining tasks are divided into two macro-categories:

- **Prediction Methods**: Aim to estimate the future or unknown value of a specific target variable starting from the values of other observed variables (predictor attributes).
- **Description Methods**: Aim to identify human-readable and interpretable patterns that describe the intrinsic structure and hidden relationships in the data.

1.4.1 Predictive Tasks: Classification

Classification aims to learn an objective function f that maps each attribute record x into a specific discrete class label $y \in \mathcal{Y}$.

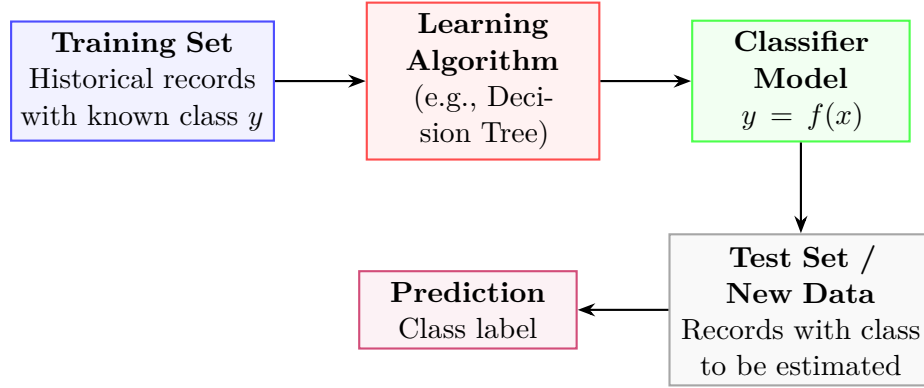


Figure 1.1: Conceptual workflow of the classification process.

The dataset is divided into two disjoint sets:

- **Training Set:** Used to train the classifier.
- **Test Set:** Used to evaluate the generalization capability of the model on unseen instances during the learning phase.

Example 1.4.1 (Didactic Example: Decision Tree Induction for Credit Worthiness). Consider a standard bank credit lending problem, where the objective is to predict whether a new applicant is financially reliable ($Credit\ Worthy \in \{Yes, No\}$).

1. Training Set (Historical data with known class labels): The lending institution has historical records of past customers with observed loan repayment outcomes:

TID	Employed	Level of Education	# Years at Address	Credit Worthy (Class)
1	Yes	Graduate	5	Yes
2	Yes	High School	2	No
3	No	Undergrad	1	No
4	Yes	High School	10	Yes

Table 1.1: Training set used to train the credit classification model.

2. Learned Model (Decision Tree): The classification algorithm extracts a hierarchical sequence of logical decisions from the historical data (Fig. 1.2):

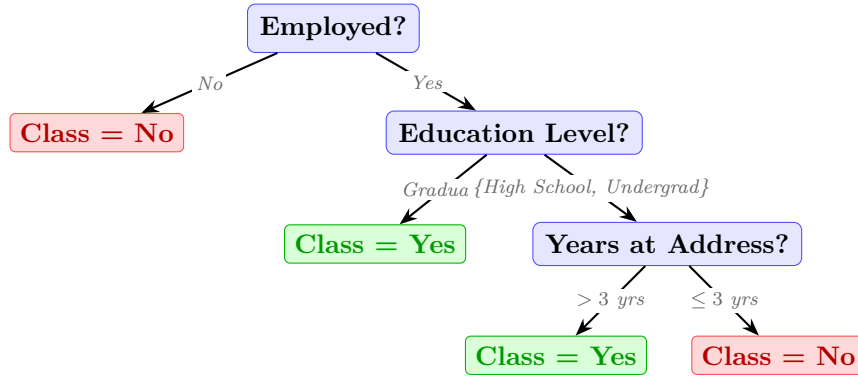


Figure 1.2: Classifier model: learned decision tree.

3. Test Set (Unseen new instances): When new credit applications arrive with unknown class labels (?), the model evaluates them by traversing the decision tree:

TID	Employed	Education	Years	Target	Model Prediction
5	Yes	Undergrad	7	?	Yes (Employed → Undergrad → $7 > 3$)
6	No	Graduate	3	?	No (Unemployed)
7	Yes	High School	2	?	No (Employed → High School → $2 \leq 3$)

Table 1.2: Evaluation of the test set using the learned decision tree.

This example highlights that the classifier does not merely memorize the training instances, but generalizes predictive logical rules applicable to any novel instance.

Example 1.4.2 (Classification Applications). Among the main and most widespread application areas of classification models we find:

- **Credit Worthiness Evaluation:** Predicting whether a loan applicant will repay the debt (*Yes/No*) based on employment, education level, and years of residence.
- **Bank Fraud Detection:** Classifying credit card transactions as legitimate or fraudulent. Features include amount, time, geographical location, and historical spending profile.
- **Churn Prediction:** Estimating the probability that a telecommunications customer will abandon the service for a competitor, analyzing call duration, data traffic volume, and recorded complaints.

- **Other applications:** Automatic categorization of news (sports, finance, politics), clinical diagnosis (benign or malignant tumor), land cover classification from satellite images.

1.4.2 Descriptive Tasks

1. Clustering

Clustering consists in organizing a set of objects into distinct groups (called *clusters*), such that:

1. The distance between objects belonging to the same cluster (**intra-cluster distance**) is **minimized** (maximum internal homogeneity).
2. The distance between objects belonging to different clusters (**inter-cluster distance**) is **maximized** (maximum external separation).

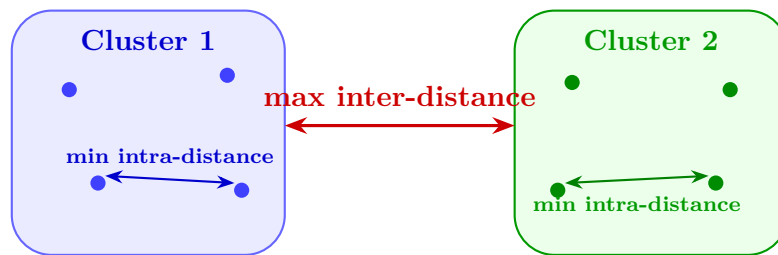


Figure 1.3: Fundamental geometric principle of clustering.

Clustering is an unsupervised technique: it does not require a priori classes.

- **Market Segmentation:** Partitioning customers by demographic traits and purchasing habits to target differentiated marketing campaigns.
- **Document Clustering:** Grouping similar textual documents based on the joint frequencies of keywords (for example, the analysis of the Enron corporate email archive).
- **Summarization and Earth Sciences:** Reducing the complexity of massive spatial series by partitioning sea surface temperature (*Sea Surface Temperature* - SST) and primary production (*Net Primary Production* - NPP) measurements in the Northern and Southern hemispheres.

2. Association Analysis (Discovery of Association Rules)

Given a set of transactions, each consisting of a subset of items taken from a global set of items, association analysis aims to discover probabilistic dependencies in the form of rules:

$$X \implies Y \quad (1.2)$$

where X and Y are disjoint itemsets ($X \cap Y = \emptyset$). The rule indicates that the presence of the items in the antecedent X systematically predicts the contextual presence of the item or items in the consequent Y .

Example 1.4.3 (Market Basket Analysis). Considering a history of cash register receipts:

TID	Purchased Items
1	Bread, Coca-Cola, Milk
2	Beer, Bread
3	Beer, Coca-Cola, Diapers, Milk
4	Beer, Bread, Diapers, Milk
5	Coca-Cola, Diapers, Milk

The algorithm extracts rules with high statistical evidence, such as:

$$\{\text{Milk}\} \implies \{\text{Coca-Cola}\}, \quad \{\text{Diapers, Milk}\} \implies \{\text{Beer}\}$$

Such information guides the strategic placement of goods on shelves and the design of cross-selling offers.

Other applications: Diagnosis of alarms in telecommunication networks (identifying temporal patterns of network alarms that trigger jointly to identify systemic root failures); medical informatics (correlation between combinations of clinical symptoms, blood tests, and emerging pathologies).

3. Anomaly / Deviation / Change Detection

It consists in identifying objects that deviate significantly from the average or expected behavior of the population. Anomalies can be generated by instrumental failures or entry errors (and in this case represent *noise* to be purged), or they can reflect exceptional events of extreme application interest (credit fraud, cyber intrusions against corporate networks, monitoring of sudden deforestation of forest cover via remote sensors).

1.5 Methodological and Computational Challenges in Data Mining

The application of traditional statistical analysis and database methods is often inadequate due to five primary challenges:

1. **Scalability:** Algorithms must handle data volumes on the order of terabytes or petabytes without computational complexity and memory footprint exploding.
2. **High Dimensionality:** The presence of hundreds or thousands of attributes exposes one to the so-called *curse of dimensionality*, where the geometric space becomes extremely empty and distances tend to lose discriminative power.
3. **Complex and Heterogeneous Data:** Need to process semi-structured or unstructured structures (graphs, texts, genomic sequences, spatiotemporal series).
4. **Data Ownership and Distribution:** Data are frequently geographically distributed across multiple organizations and subject to strict legal privacy and security constraints.
5. **Non-Traditional Analysis:** Need to extract knowledge from continuous streams in real-time (*data stream mining*) without the possibility of storing the entire dataset.

Chapter 2

Data Understanding and Data Exploration

Practical Lab: Interactive Execution on Jupyter Notebook

The Python code snippets presented in this chapter correspond to the companion lab notebook:

- **Notebook:** `1_basics_and_understanding.ipynb`
- **Direct access:** [[Open in Jupyter Lab](http://localhost:8888)] (`http://localhost:8888`)
- **Quickstart container:** Run from root directory: `docker compose up jupyter`
- **Local workspace folder:** `Data-Mining/notebooks/` (datasets located in `./data_notebook/`).

2.1 Objectives of Data Understanding

The data understanding phase (*Data Understanding*) constitutes an indispensable prerequisite for any Data Mining activity. Before selecting or applying any algorithm, the analyst must acquire an overall view of the structure, distribution, and intrinsic quality of the dataset with respect to the project objectives.

In this preliminary phase, it is necessary to answer crucial methodological questions:

- What types of attributes compose the dataset?

- What is the overall level of data quality (presence of noise, errors, or duplicates)?
- How are the numerical and categorical variables distributed?
- Are there evident correlations or redundancies among the attributes?
- How are outliers structured and what application meaning do they assume?
- What strategies should be adopted for missing values?
- Is it necessary to derive or transform attributes to make mining more effective?

2.2 Types of Datasets

The collected data present themselves in different logical-geometric structures based on the nature of the application domain:

1. Record Data:

- **Data Matrix:** A set of objects described by the same fixed set of numerical attributes. A dataset of m objects and n attributes is formally represented by an $m \times n$ matrix:

$$\mathbf{X} = \begin{pmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,n} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m,1} & x_{m,2} & \cdots & x_{m,n} \end{pmatrix} \quad (2.1)$$

In this formalization, each object x_i is a point in an n -dimensional vector space $\mathbb{R}^n$.

- **Document Data (Bag-of-Words):** Each textual document is represented by a term vector (*term vector*). Each component of the vector corresponds to a word in the vocabulary, and the associated value indicates its frequency of occurrence in the text.
- **Transaction Data:** Each record represents a transaction (identified by a TID) and contains a variable set of purchased items or observed events.

2. **Graph Data:** Structures composed of nodes and edges that model relationships and connectivity. Examples include the network of hyper-text links of the World Wide Web, social network graphs, and molecular structures (for example, the planar molecule of benzene C_6H_6).
3. **Ordered Data:**
 - **Sequences of transactions:** Temporal successions of purchases or events for each user.
 - **Genomic data:** Discrete biological sequences composed of nucleotide characters $\{A, C, G, T\}$.
 - **Spatio-Temporal Data:** Continuous or sampled measurements over geographical coordinates and regular time intervals (for example, the monthly average temperature of the land and ocean surface).

2.3 Objects and Attributes

A dataset is a collection of **data objects** and their corresponding **attributes**.

- An **object** (also known as a record, point, case, sample, entity, or instance) represents a single entity of the domain of interest.
- An **attribute** (also known as a variable, field, characteristic, dimension, or feature) represents a measurable property or characteristic of an object (for example, age, taxable income, or eye color).

2.3.1 Measurement Scales of Attributes

The type of measurement scale of an attribute is defined by the set of algebraic properties and mathematical operators that preserve the meaning of the measurement:

1. **Distinctness** ($=, \neq$): Possibility to distinguish whether two values are equal or different.
2. **Order** ($<, >$): Possibility to establish an order relationship between values.
3. **Meaningful differences** ($+, -$): Presence of a constant unit of measurement that allows quantifying the interval between two values (zero is conventional/arbitrary).

4. **Meaningful ratios** (*, /): Presence of an **intrinsic absolute zero** (indicating the total absence of the measured quantity), which makes the proportional comparison between quantities meaningful.

Scale	Properties	Description	Examples	Statistical Operators
Nominal	Distinctness (=, $\neq$)	Qualitative values or categorical labels without intrinsic ordering	SSN, ZIP code, eye color, gender	Mode, entropy, χ^2 test, contingency
Ordinal	Distinctness and Order (=, $\neq$, $<$, $>$)	Values with meaningful ordering but non-constant interval	Educational degree, Likert scale (1–10), mineral hardness	Median, percentiles, rank correlation (Spearman)
Interval	Order and Differences (+, $-$)	Quantitative differences with meaning; zero is conventional	Temperature in $^{\circ}\text{C}$ or $^{\circ}\text{F}$, calendar dates	Mean, variance, standard deviation, Pearson, t and F tests
Ratio	All properties (=, $<$, $+$, $*$, $/$)	Presence of absolute zero; ratios between physically valid measurements	Age, income, mass, counts, Kelvin	Geometric and harmonic mean, percentage variation

Table 2.1: Taxonomy of attribute measurement scales (Stevens).

2.3.2 Further Distinctions of Attributes

- **Binary Attributes:** Nominal attributes that exclusively admit two states (0 and 1). They are distinguished into:
 - *Symmetric Binary:* Both outcomes have equal informational importance (for example, biological gender).
 - *Asymmetric Binary:* One of the two outcomes is rare but of extreme informational relevance (for example, a positive outcome to a diagnostic test for a rare disease). By convention, the value 1 is associated with the event of greater interest.
- **Discrete vs Continuous Attributes:**

- *Discrete*: Assume a finite or countably infinite set of values (typically represented by integers, counts, or categories). Binary attributes constitute a special case of discrete attribute.
- *Continuous*: Assume values in a continuous subset of real numbers $\mathbb{R}$ (temperature, weight, coordinates). In computational practice, they are measured and stored using floating-point representations (*floating-point*) with finite precision.

2.4 Data Quality

Data quality is a critical factor for the success of any analytical initiative. As highlighted in Thomas Redman's studies, problems arising from poor quality data typically cost organizations between 10% and 20% of their overall revenues. If a credit risk model is trained on flawed or erroneous data, the system will reject solvent debtors and disburse loans to insolvent individuals.

2.4.1 Dimensions of Data Quality

Quality problems manifest themselves under various dimensions:

1. **Syntactic Accuracy:** The recorded value violates formal constraints or does not belong to the predetermined domain (for example, the string "female" for gender, or the presence of letters in a numeric field). Such anomalies can be identified immediately and automatically through simple schema and type checks.
2. **Semantic Accuracy:** The data is formally correct and conforms to the expected type, but false in the reference reality (for example, associating the name "John Smith" with the female gender or recording a biological age of 180 years). Detecting these discrepancies requires domain rules (*business rules*) or cross-referencing with certified external sources.
3. **Completeness:** Systematic lack of values for certain attributes or absence of entire records. An insidious violation of completeness is **sampling bias** (for example, the analysis of insolvency risk carried out on a historical archive that only includes customers already approved by the bank, excluding a priori rejected low-income profiles).
4. **Unbalanced Data:** Strongly asymmetric distribution of classes (for example, fraudulent transactions equal to 0.1% of the total or defective products on highly automated production lines).

5. **Timeliness:** Obsolete degree of data updating compared to the current dynamics of the observed phenomenon.
6. **Duplicate Data:** Presence of identical or nearly identical records relating to the same entity (for example, the same customer registered multiple times with different email addresses following the merger of two heterogeneous databases). Such issues require dedicated deduplication and *record linkage* processes.

Anomalies in Measurement Streams and Sensors

In industrial monitoring and IoT systems, a broken or blocked sensor typically transmits a flat, constant value of zero. During the exploratory phase, such readings must not be treated as genuine physical measurements, but rather interpreted and handled in all respects as **missing values** (caused by hardware malfunction).

2.5 Descriptive Statistics and Univariate Exploration

2.5.1 Measures of Central Tendency

- **Sample Mean ($\bar{x}$):** For a sample of size m :

$$\bar{x} = \frac{1}{m} \sum_{i=1}^m x_i \quad (2.2)$$

The mean is a distributive measure (it can be calculated by partitioning the dataset into subsets), but it is **extremely sensitive to the presence of outliers**.

- **Median:** The value that occupies the central position in the increasing order of the observations (or the mean of the two central values if m is even). It is a robust measure, with a breakdown point (*breakdown point*) of 50%.
- **Trimmed Mean:** Mean calculated by excluding a fixed percentage (for example 5% or 10%) of the most extreme values at both ends.
- **Mode:** The value or category that occurs with the maximum absolute frequency. A distribution can be unimodal, bimodal, multimodal, or modeless if all observations appear only once.

2.5.2 Skewness and Multimodality

- **Positively Skewed / Right-skewed:** The right tail of the distribution is elongated. Most observations are concentrated on values lower than the arithmetic mean (Mode < Median < Mean). A typical example is the distribution of **incomes**: the vast majority of citizens earn low or medium incomes, while a small number of individuals earn extremely high incomes, pulling the mean upwards.
- **Bimodal Distribution:** Presence of two distinct peaks in the frequency density. It typically signals the influence of an unstated binary latent variable (for example, the distribution of human body weight in a mixed population, influenced by the binary variable sex).

2.5.3 Measures of Dispersion

- **Range:** Difference between maximum value and minimum value:
Range = $x_{\max} - x_{\min}$.
- **Sample Variance (s^2) and Standard Deviation (s):**

$$s^2 = \frac{1}{m-1} \sum_{i=1}^m (x_i - \bar{x})^2, \quad s = \sqrt{s^2} \quad (2.3)$$

The standard deviation measures the dispersion around the mean; it is zero only if all observations are identical ($s = 0$), otherwise $s > 0$. It is also sensitive to outliers.

- **Robust Measures of Dispersion:**

– *Absolute Average Deviation* (AAD):

$$\text{AAD} = \frac{1}{m} \sum_{i=1}^m |x_i - \bar{x}| \quad (2.4)$$

– *Median Absolute Deviation* (MAD):

$$\text{MAD} = \text{median}(|x_i - \text{median}(x)|) \quad (2.5)$$

– *Interquartile Range* (IQR): Distance between the third quartile Q_3 (75th percentile) and the first quartile Q_1 (25th percentile):

$$\text{IQR} = Q_3 - Q_1 \quad (2.6)$$

2.5.4 Box Plot and Five-Number Summary

The **Box Plot** (box and whisker plot) offers a non-parametric visual summary of the distribution based on the **five-number summary**:

$$[\text{Minimum}, Q_1, \text{Median} (Q_2), Q_3, \text{Maximum}]$$

Its geometric structure relies on the concept of quantile: given $p \in (0, 100)$, the $p\%$ -**quantile** (or p^{th} percentile) is the cutoff value x_p such that $p\%$ of sample observations are smaller than or equal to x_p , while $(100 - p)\%$ are greater. Specifically:

- Q_1 corresponds to the 25%-quantile (first quartile).
- The median Q_2 corresponds to the 50%-quantile (second quartile).
- Q_3 corresponds to the 75%-quantile (third quartile).
- The width of the central box represents the interquartile range $\text{IQR} = Q_3 - Q_1$, encompassing the central 50% of the data mass.

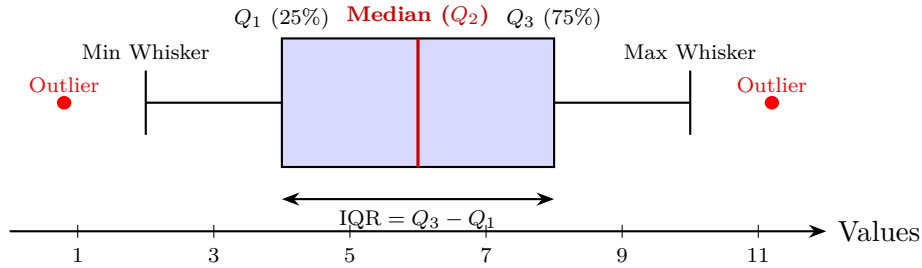


Figure 2.1: Geometric anatomy of a Box Plot with identification of outliers.

The limits of the whiskers are conventionally calculated by extending up to the most extreme observed value within the interval $[Q_1 - 1.5 \times \text{IQR}, Q_3 + 1.5 \times \text{IQR}]$. The observations that fall outside such thresholds are visually labeled as **outliers**.

A key extension for bivariate and multivariate exploration is given by **Conditional Box Plots**: the distribution of a continuous numerical attribute is stratified and displayed by placing box plots side by side for each category of a discrete attribute (typically the target class or a natural clustering). Figure 2.2 illustrates the distribution of *Petal Length* stratified across the three flower species of the Iris dataset, highlighting that *Setosa* is distinctly separable with very low dispersion, whereas *Versicolor* and *Virginica* exhibit a slight overlap in their interquartile ranges.

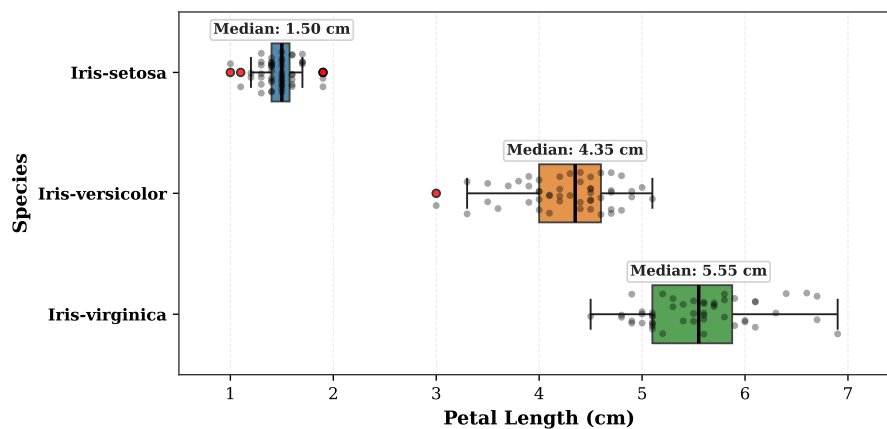


Figure 2.2: Conditional Box Plot computed on the real Iris dataset: empirical distribution of *Petal Length* stratified across the three flower species, displaying median, quartiles, and individual observations.

2.5.5 Python Lab: Descriptive Inspection, IQR Calculation, and Box Plots

Run this section in the Interactive Notebook

Notebook: [1_basics_and_understanding.ipynb](#) [Open in Jupyter Lab]

Local server: <http://localhost:8888> (launch command: `docker compose up jupyter`)

In the companion notebook `1_basics_and_understanding.ipynb`, univariate inspection, summary statistics, and Tukey outlier boundaries are implemented using Pandas and Seaborn as shown in Listing 2.1:

Listing 2.1: Univariate inspection, IQR calculation, and Box Plot visualization with Pandas and Seaborn.

```

1 import pandas as pd
2 import numpy as np
3 import seaborn as sns
4 import matplotlib.pyplot as plt
5
6 # 1. Load the restaurant tips dataset
7 data_path = './data_notebook/data/'
8 df = pd.read_csv(f'{data_path}correct_tips.csv',
    index_col=0)

```

```

9
10 # 2. Comprehensive descriptive statistics
11 print(df.describe().T[['mean', 'std', 'min', '25%', '50%',
12                        '75%', 'max']])
13
14 # 3. Analytical IQR computation and Tukey whisker
15     boundaries for 'tip'
16 q25 = df['tip'].quantile(0.25)
17 q75 = df['tip'].quantile(0.75)
18 iqr = q75 - q25
19 lower_whisker = q25 - 1.5 * iqr
20 upper_whisker = q75 + 1.5 * iqr
21
22 outliers = df[(df['tip'] < lower_whisker) | (df['tip'] >
23              upper_whisker)]
24 print(f"IQR = {iqr:.2f} | Whiskers: [{lower_whisker:.2f}, {upper_whisker:.2f}] | Outliers detected: {len(outliers)}")
25
26 # 4. Univariate and conditional box plot rendering
27 fig, axes = plt.subplots(1, 2, figsize=(12, 4))
28 df.boxplot(column=['tip'], ax=axes[0])
29 axes[0].set_title("Univariate Box Plot (Tip)")
30
31 sns.boxplot(data=df, x='sex', y='tip', hue='time', ax=
32             axes[1])
33 axes[1].set_title("Conditional Box Plot (Tip by Sex and
34                  Time)")
35 plt.tight_layout()
36 plt.show()

```

Critical Comparison: Box Plot vs Histogram

Although the box plot effectively synthesizes position and dispersion, **it is not able to show the real shape of the distribution**. Two distributions having radically different densities (for example, a uniform unimodal distribution and a strongly bimodal distribution with an empty central valley) can produce identical values of Q_1 , median, Q_3 , Minimum, and Maximum, and therefore generate the exact same box plot. The histogram is essential to detect multimodality.

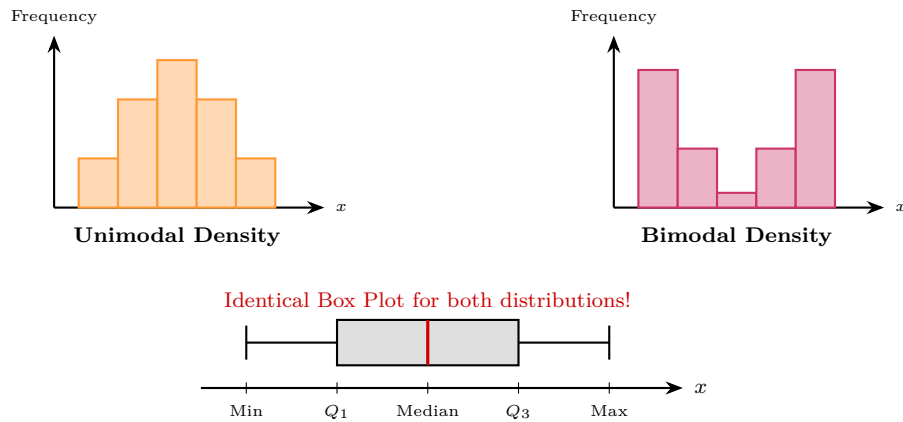


Figure 2.3: Critical comparison: two distributions with diametrically opposite behaviors (unimodal vs bimodal with empty center) generate the same five-number summary and the same Box Plot.

2.5.6 Bar Chart for Categorical Attributes

The **Bar Chart** visually represents the frequency distribution of a qualitative or categorical attribute. Each discrete category is represented by a vertical (or horizontal) bar, whose height corresponds to the absolute count or relative frequency. Unlike histograms, bars are separated by empty spaces to highlight that the modalities do not constitute a numerical continuum.

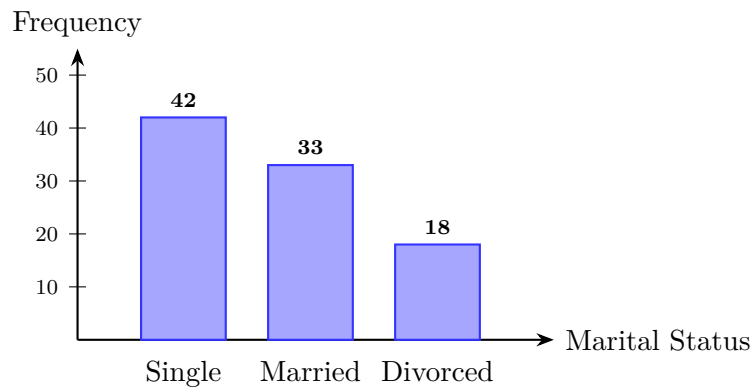


Figure 2.4: Example of a Bar Chart for a nominal categorical attribute (Marital Status).

2.5.7 Histograms for Numerical Attributes and Choice of Bins

A **histogram** shows the empirical frequency distribution of a continuous numerical attribute. The domain of variability of the quantity is partitioned (*discretized*) into a predetermined number k of contiguous intervals called *bins* (or class widths). For each bin, the height of the overlying bar is proportional to the number of sample observations that fall within it.

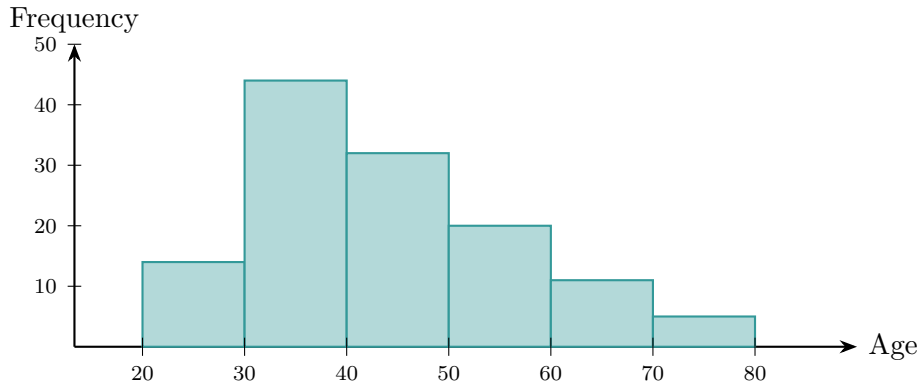


Figure 2.5: Example of a Histogram for a continuous numerical attribute with contiguous bins of constant width.

An incorrect choice of the number of bins profoundly alters the perception of the data:

- An insufficient number of bins causes *oversmoothing*, leveling the distribution and masking skewnesses and multimodality.
- An excessive number of bins causes *undersmoothing*, fragmenting the sample into discontinuous frequencies and emphasizing sampling noise.

For approximately normal unimodal distributions and moderate sample sizes n , the standard estimate for the optimal number of intervals is given by **Sturges' rule**:

$$k = 1 + \lceil \log_2(n) \rceil = 1 + \lceil 3.322 \log_{10}(n) \rceil \quad (2.7)$$

where n represents the sample cardinality. This heuristic explicitly assumes a Gaussian-like distribution. When the sample size grows considerably or the underlying distribution exhibits multiple modes or heavy skewness, Sturges' rule tends to generate an insufficient number of bins; in such cases, it is

crucial to test alternative values of k experimentally (e.g., comparing low-, medium-, and high-resolution partitionings such as 5, 17, or 50 bins) to assess the robustness of observed patterns.

2.6 Multivariate Visualization Techniques

2.6.1 Scatter Plot and Multi-Dimensional Visualization

The **Scatter Plot** treats pairs of numerical attributes as Cartesian coordinates (x_i, y_i) in the plane. It is the main visual tool for:

- Identifying linear correlations or non-linear functional trends.
- Detecting natural clusters of points (*clusters*).
- Finding multivariate outliers not visible in univariate projections.

Example 2.6.1 (Analysis of the Anderson/Fisher Iris Dataset). The famous *Iris* dataset (150 samples of flowers belonging to three species: *Setosa*, *Versicolor*, *Virginica*) describes 4 numerical attributes: sepal length and width, petal length and width. The bivariate scatter plot shows that the attributes related to the petal (*Petal Length* vs *Petal Width*) guarantee a clear separation among the species, also allowing the isolation of potential anomalies.

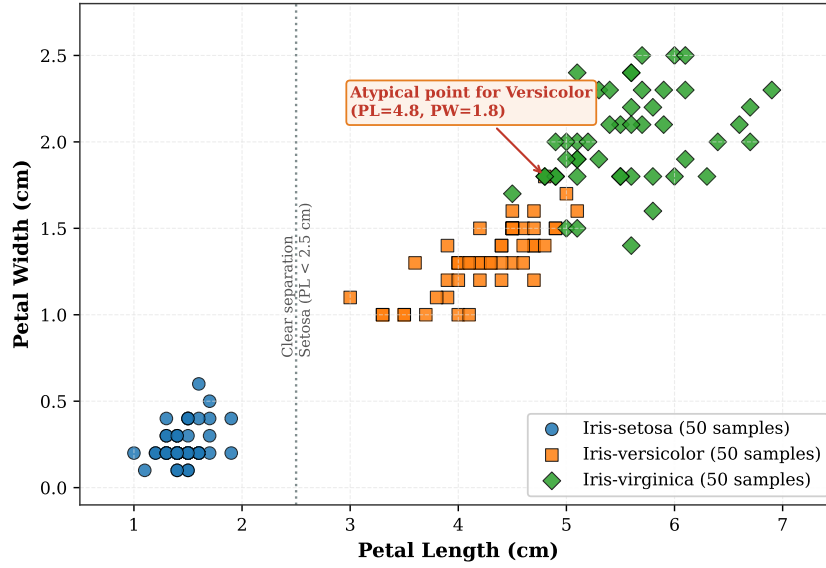


Figure 2.6: Real bivariate Scatter Plot (*Petal Length* vs *Petal Width*) across all 150 samples of the Iris dataset, featuring class-based color and geometric encoding, and visual confirmation of the clear separation of *Setosa* and boundary atypical points.

To simultaneously explore all pairs of numerical attributes, a **Scatter Plot Matrix** (SPLOM) is used, which is an $n \times n$ grid where each off-diagonal cell (i, j) displays the scatter plot between attribute i and attribute j , while the main diagonal hosts the histograms or univariate density estimations.

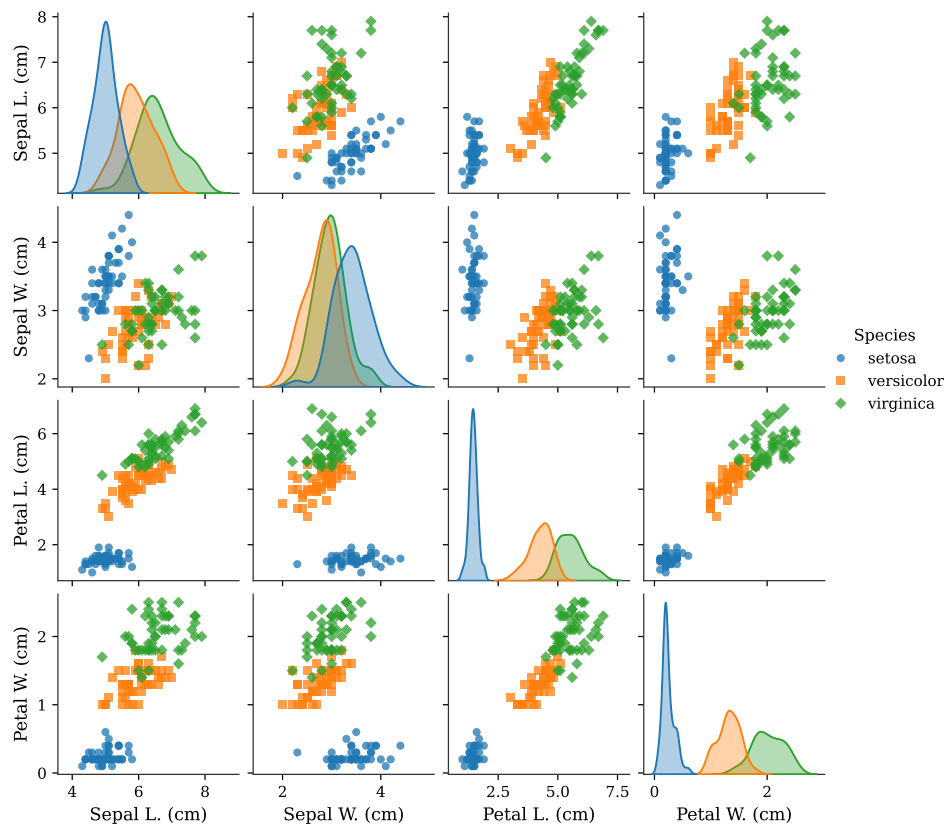


Figure 2.7: Complete Scatter Plot Matrix (SPLOM) computed across all 150 samples of the Iris dataset for all 4 continuous attributes, showing univariate kernel density estimations (KDE) along the main diagonal and bivariate scatter plots in the outer cells.

2.6.2 Python Lab: Multivariate Scatter Plots and Scatter Matrix

Run this section in the Interactive Notebook

Notebook: `1_basics_and_understanding.ipynb`
[Jupyter Lab](#)

[Open in

Local server: <http://localhost:8888> (launch command: `docker compose up jupyter`)

Bivariate visual representations and the Scatter Plot Matrix are generated with Seaborn and Pandas as illustrated in Listing 2.2:

Listing 2.2: Rendering bivariate Scatter Plots and a Scatter Plot Matrix with Seaborn and Pandas.

```
1 import pandas as pd
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4
5 df = pd.read_csv('./data_notebook/data/correct_tips.csv',
6                 , index_col=0)
7
8 # 1. Bivariate scatter plot with color and style
9 encoding
10 plt.figure(figsize=(7, 4))
11 sns.scatterplot(data=df, x='total_bill', y='tip', hue='
12                 time', style='sex', s=60)
13 plt.title("Total Bill vs Tip (Stratified by Time and Sex
14           )")
15 plt.xlabel("Total Bill ($)")
16 plt.ylabel("Tip ($)")
17 plt.show()
18
19 # 2. Scatter Plot Matrix (SPLOM) across numerical
20 features
21 numeric_features = ['total_bill', 'tip', 'size', 'age']
22 pd.plotting.scatter_matrix(df[numeric_features], figsize
23                             =(8, 8), diagonal='kde', alpha=0.7)
24 plt.suptitle("Scatter Plot Matrix with Diagonal KDE
25               Estimates", y=1.02)
26 plt.show()
```

2.6.3 Visualization as a Test and Exploratory Heuristic

Visual inspection serves as an irreplaceable preliminary test on data structure, whose sound interpretation requires precise epistemological awareness:

- **Presence of visual evidence:** When visualizations expose distinct patterns, separated clusters, or isolated exceptions (*outliers*), there is empirical certainty that genuine structural properties or real phenomena exist in the dataset.
- **Absence of visual evidence does not imply absence of structure:** Conversely, when conventional visualizations display no obvious arrangement or appear purely random, one **cannot** infer that the data

is devoid of relationships. Complex multivariate dependencies, subtle non-linear associations, or high-order interactions involving four or more attributes simultaneously elude two-dimensional projections, requiring higher-dimensional projections or analytical modeling techniques.

2.6.4 High-Dimensionality Visualization

Parallel Coordinates

When the number of dimensions exceeds 3, the conventional Cartesian representation becomes inapplicable. **Parallel Coordinates** replace the orthogonal axes with a set of equidistant vertical parallel axes, each corresponding to an attribute.

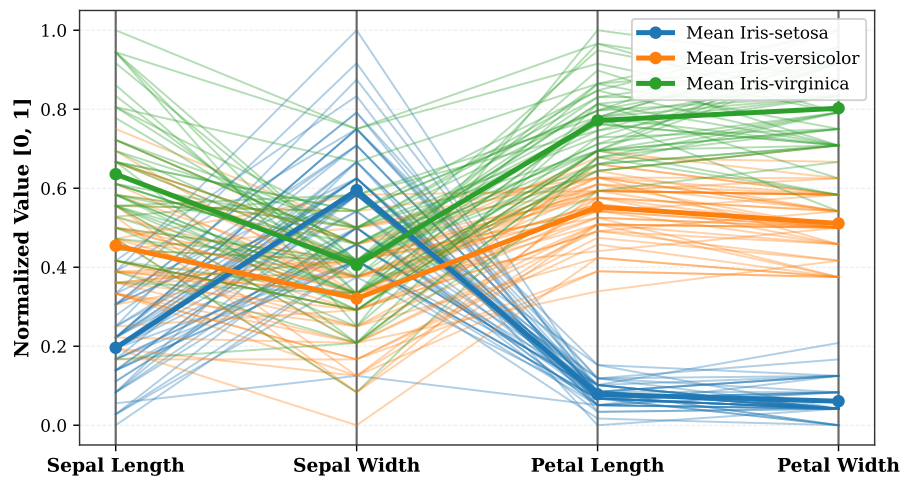


Figure 2.8: 4-dimensional representation using Parallel Coordinates computed on the real Iris dataset (150 samples normalized to $[0, 1]$): thin curves depict individual flower samples, while thick lines show the distinctive mean profiles per species.

- Each multidimensional object is represented by a **polygonal broken line** that connects the respective scaled values along the vertical axes.
- Objects belonging to the same class tend to form bundles of parallel lines or with coordinated trends.
- **Sensitivity to the ordering of the axes:** The visibility of clusters and correlations depends critically on the sequential arrangement of the axes. Two strongly correlated attributes show regular geometric patterns only if positioned as adjacent axes.

Radar Plot and Star Plot

In **Radar Plots**, the axes start from a common center and radiate out in a star shape at constant angular intervals. The value of each attribute defines a vertex along its own radius; joining the vertices yields a polygon for each instance. **Star Plots** apply the same principle by drawing each object in a separate graph, facilitating comparison for individual profiles.

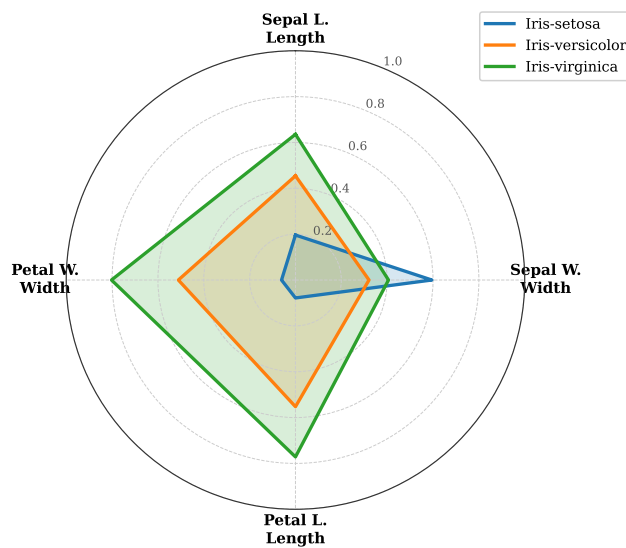


Figure 2.9: Empirical multivariate Radar Plot computed on the normalized mean profiles of the three Iris species across the 4 morphological attributes.

2.7 Linear Correlation and Association Analysis

Correlation analysis aims to identify, quantify, and interpret the strength and direction of the linear relationship between pairs of continuous numerical attributes.

2.7.1 Motivations and Multicollinearity in Predictive Models

Within the data understanding workflow, assessing correlation between variables serves several methodological goals:

- **Identifying informative redundancies:** If two attributes exhibit nearly identical trajectories, one of them can be discarded without appreciable loss of descriptive power, reducing the dimensionality of the feature space.
- **Preparation for predictive models and clustering:** Many machine learning algorithms (such as linear regression, logistic regression, or distance-based clustering) assume independent features or benefit substantially from the removal of collinear attributes.
- **Detecting multicollinearity:** Occurs when two or more predictor variables (*features*) in a regression model are strongly linearly related to one another. Consider the paradigmatic example of predicting house prices using simultaneously *surface area in square metres* (m^2) and *surface area in square feet* (ft^2). Both variables convey exactly the same information up to a constant conversion factor. Consequently, the regression model experiences severe numerical instability, struggling to disentangle the individual marginal contributions of each predictor and inflating estimator variance.

2.7.2 From Variance to Sample Covariance

While sample variance quantifies the dispersion of observations for a single variable around its own mean:

$$s_x^2 = \text{Var}(X) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2 \quad (2.8)$$

in data mining applications interest is constantly focused on pairwise interactions between attributes:

- Do customers with higher taxable incomes spend larger sums?
- Does a product's unit price lead to a contraction in sales volume?
- Do students who study more weekly hours systematically attain higher exam grades?
- Do two physical sensors arranged in parallel provide largely redundant measurement streams?

To quantify how two numerical quantities vary jointly, we introduce **sample covariance**:

Definition 2.7.1 (Sample Covariance). Given a sample of n observation pairs (x_i, y_i) , the sample covariance between X and Y is defined as:

$$\text{Cov}(X, Y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}) \quad (2.9)$$

where $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$ and $\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$ represent the respective sample means.

A fundamental theoretical property dictates that variance is simply the covariance of a variable with itself:

$$\text{Cov}(X, X) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2 = \text{Var}(X) = s_x^2 \quad (2.10)$$

2.7.3 Sign Origin and Geometric Quadrant Interpretation

The algebraic sign of each term in the summation stems from the product of paired deviations from their respective means:

- **Positive concordant deviations:** $(x_i - \bar{x}) > 0$ and $(y_i - \bar{y}) > 0 \implies (+) \times (+) = +$.
- **Negative concordant deviations:** $(x_i - \bar{x}) < 0$ and $(y_i - \bar{y}) < 0 \implies (-) \times (-) = +$.
- **Discordant deviations:** $(+) \times (-) = -$ or $(-) \times (+) = -$.

Translating the Cartesian origin to the sample centroid $(\bar{x}, \bar{y})$, the two-dimensional space is divided into four quadrants (Figure 2.10):

1. If observations in quadrants **I and III** (concordant deviations) dominate, positive products prevail and the sample covariance is strictly positive ($\text{Cov}(X, Y) > 0$).
2. If quadrants **II and IV** (discordant deviations) prevail, negative products dominate and the covariance is strictly negative ($\text{Cov}(X, Y) < 0$).
3. If observations are uniformly or symmetrically scattered across all four quadrants, positive and negative terms cancel each other out, resulting in a covariance close to zero ($\text{Cov}(X, Y) \approx 0$).

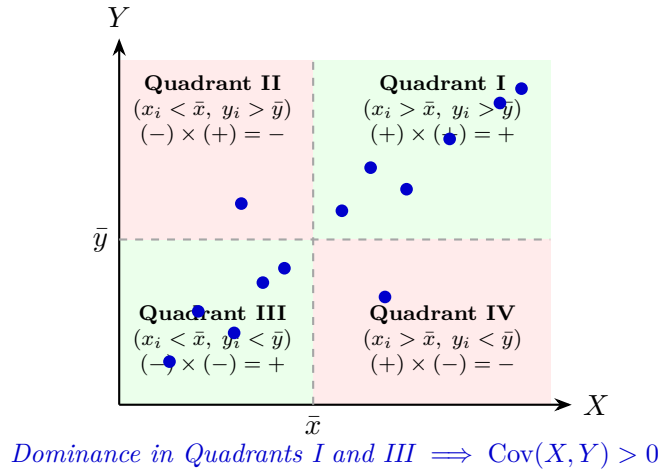


Figure 2.10: Geometric interpretation of covariance across quadrants centered at $(\bar{x}, \bar{y})$. Observations in concordant quadrants I and III yield positive contributions, whereas discordant quadrants II and IV yield negative contributions.

2.7.4 Limitation of Covariance: Scale and Unit Dependence

Although covariance unambiguously reveals the direction of linear agreement, it suffers from a critical drawback: **it depends strictly on the measurement units of the variables**. Consider the relationship between annual income and household expenditure:

- If both income and expenditure are measured in euros, the unit of $\text{Cov}(\text{Income}, \text{Expenditure})$ is euro^2 .
- If income is converted into *thousands of euros*, the numerical covariance drops by a factor of 1000, even though the substantive economic relationship between both quantities remains strictly unchanged.

Consequently, covariance indicates whether the linear trend is positive or negative, but its absolute magnitude cannot be compared across different scales or attribute pairs without standardization.

2.7.5 Pearson Linear Correlation Coefficient

To obtain a unit-free, standardized metric, sample covariance is divided by the product of the respective sample standard deviations:

Definition 2.7.2 (Pearson Correlation Coefficient). Given a sample of n pairs (x_i, y_i) , the sample linear correlation coefficient r_{xy} is defined as:

$$r_{xy} = \frac{\text{Cov}(X, Y)}{s_x s_y} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2}} \quad (2.11)$$

where s_x, s_y are the sample standard deviations of X and Y .

Core properties of the Pearson correlation coefficient:

- **Dimensionless and scale-invariant:** r_{xy} is a pure number, invariant under any positive affine transformation ($X' = aX + b$, with $a > 0$).
- **Bounded range:** $-1 \leq r_{xy} \leq +1$.
- **Perfect collinearity** ($|r_{xy}| = 1$): Points lie precisely on a straight line; with positive slope if $r_{xy} = +1$, and negative slope if $r_{xy} = -1$.
- **Absence of linear association** ($r_{xy} \approx 0$): Signals little to no linear relationship.
- **Blindness to non-linear dependencies:** As illustrated in Figure 2.11(d), zero correlation ($r_{xy} \approx 0$) does not imply statistical independence: deterministic quadratic or sinusoidal relationships may exhibit exactly zero Pearson correlation.

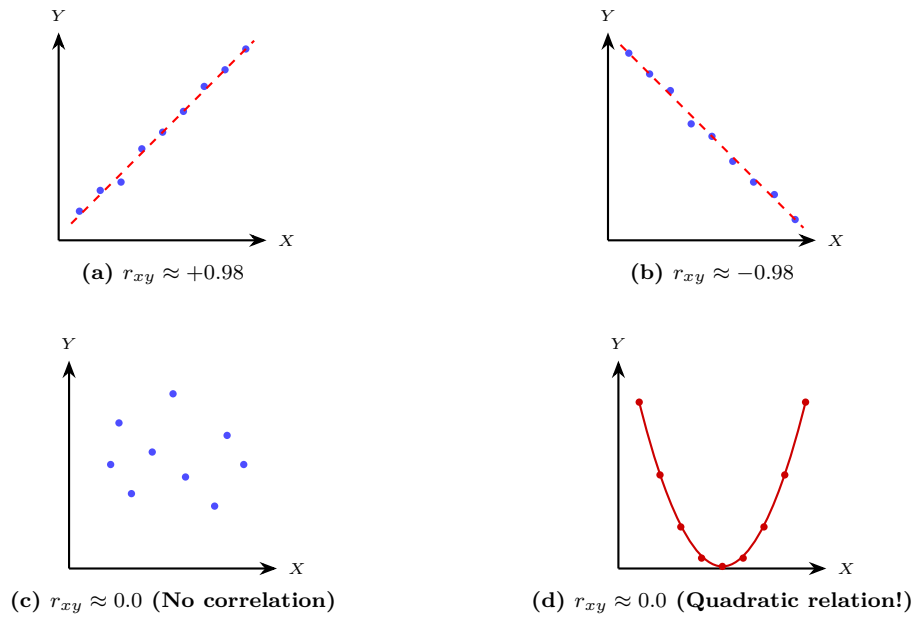


Figure 2.11: Visual interpretation of the Pearson correlation coefficient: (a) positive correlation; (b) negative correlation; (c) absence of linear relation; (d) non-linear deterministic dependence missed by Pearson.

2.7.6 Statistical Association vs Causation and Confounding Factors

A fundamental tenet in data science is the epistemological boundary separating association from causation:

$$\text{Correlation} \neq \text{Causation} \quad (2.12)$$

Consider an academic dataset where weekly study hours and examination marks exhibit a high positive correlation ($r_{xy} = +0.98$). Students with fewer study hours tend to score below the mean, while those studying longer hours score above it. Although this demonstrates an undeniably strong positive linear association, **it does not warrant the conclusion that 2 additional study hours will deterministically cause an increase in marks**. Multiple confounding variables (such as prior academic background, lecture attendance, or classroom attention) may influence both quantities simultaneously.

Linear correlation:

1. Does not identify the causal direction ($X \rightarrow Y$ or $Y \rightarrow X$).

2. Can be driven by a **confounding variable** (omitted variable bias).

Example 2.7.1 (The Confounding Variable: Ice Cream and Drowning). In seasonal regional records, ice cream sales and drowning incidents show a strong positive correlation. Eating ice cream clearly does not cause drowning. The observed relationship is driven entirely by an omitted confounding variable: **high summer temperatures** (heatwaves), which simultaneously increase demand for chilled desserts and the number of people swimming, producing a spurious association.

2.7.7 Spearman's Rank Correlation for Monotonic Relationships

When the relationship between two variables is monotonic but non-linear, or in the presence of severe outliers that distort means and standard deviations, the **Spearman rank correlation coefficient** (ρ_s) is preferred.

Spearman does not evaluate original continuous values directly, but replaces each value with its **rank** in ascending order:

Definition 2.7.3 (Spearman's Rank Correlation Coefficient). Let $\text{rank}(x_i)$ and $\text{rank}(y_i)$ denote the ranks of x_i and y_i . If there are no ties, the coefficient is given by:

$$\rho_s = 1 - \frac{6 \sum_{i=1}^n d_i^2}{n(n^2 - 1)}, \quad \text{with } d_i = \text{rank}(x_i) - \text{rank}(y_i) \quad (2.13)$$

In the presence of tied ranks, ρ_s is formally equivalent to Pearson's correlation computed directly on the rank vectors.

Essential properties:

- $\rho_s = +1$ when the rankings of both variables are in exactly the same order (**monotonically increasing** relationship).
- $\rho_s = -1$ when the rankings are in reverse order (**monotonically decreasing** relationship).
- Measures whether variables tend to move in the same or opposite direction regardless of linearity (e.g., for $Y = X^3$ or $Y = \log(X)$, $\rho_s = 1$).
- Highly robust against isolated extreme outliers.

2.7.8 Correlation Matrix and Feature Selection

In datasets with multiple continuous attributes, all pairwise bivariate correlations are systematically arranged in a symmetric $p \times p$ **correlation matrix**. In the Iris dataset (Figure 2.12), for example, the matrix reveals an exceptionally strong linear and monotonic correlation between *Petal Length* and *Petal Width* ($r = 0.96$, $\rho_s = 0.94$), as well as between *Sepal Length* and *Petal Length* ($r = 0.87$, $\rho_s = 0.88$), signaling significant feature redundancy and suggesting that dimensionality reduction or feature selection can discard or combine one of the two without sacrificing model performance.

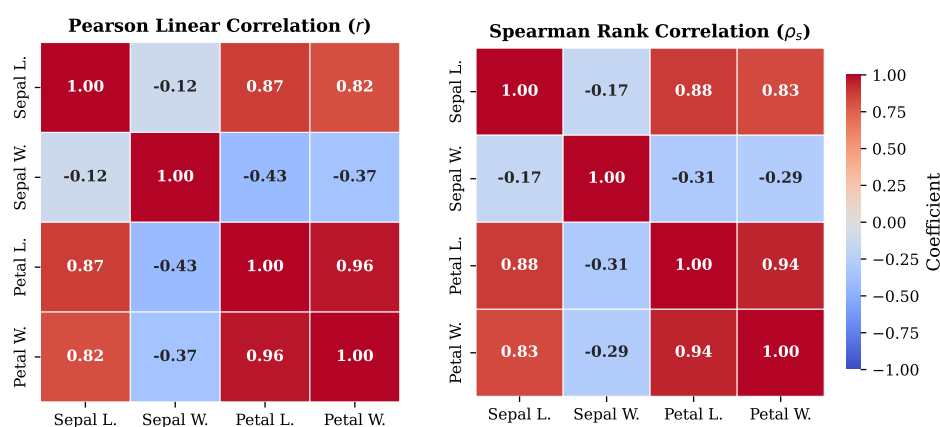


Figure 2.12: Bivariate correlation matrices computed on the real Iris dataset: Pearson linear correlation (r , left) and Spearman rank correlation (ρ_s , right). The visualization clearly shows the strong positive co-dependency among petal attributes and the moderate negative correlation involving sepal width.

2.7.9 Python Lab: Correlation Matrices and Heatmap Visualization

Run this section in the Interactive Notebook

Notebook: [1_basics_and_understanding.ipynb](#) [Open in Jupyter Lab]

Local server: <http://localhost:8888> (launch command: `docker compose up jupyter`)

In the companion notebook `1_basics_and_understanding.ipynb`, comparative correlation coefficients (Pearson, Spearman, Kendall) and annotated heatmaps are produced as shown in Listing 2.3:

Listing 2.3: Comparative correlation matrix calculation (Pearson, Spearman, Kendall) and Heatmaps with Seaborn.

```
1 import pandas as pd
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4
5 df = pd.read_csv('./data_notebook/data/correct_tips.csv',
6                  , index_col=0)
7
8 # 1. Compute pairwise correlation matrices across
9    numerical attributes
10 corr_pearson = df.corr(numeric_only=True, method='
11    pearson') # linear
12 corr_spearman = df.corr(numeric_only=True, method='
13    spearman') # monotonic
14 corr_kendall = df.corr(numeric_only=True, method='
15    kendall') # rank concordance
16
17 # 2. Side-by-side comparative heatmap visualization
18 fig, axes = plt.subplots(1, 2, figsize=(13, 5))
19 sns.heatmap(corr_pearson, annot=True, fmt=".2f", cmap="
20    coolwarm", vmin=-1, vmax=1, ax=axes[0])
21 axes[0].set_title("Pearson Linear Correlation (r)")
22
23 sns.heatmap(corr_spearman, annot=True, fmt=".2f", cmap="
24    coolwarm", vmin=-1, vmax=1, ax=axes[1])
25 axes[1].set_title("Spearman Rank Correlation (rho_s)")
26 plt.tight_layout()
27 plt.show()
```

2.8 Handling Outliers and Missing Values

2.8.1 Outliers: Noise vs Analytical Target

An **outlier** is a data object whose characteristics deviate considerably from the vast majority of observations in the dataset:

- **Outliers as noise:** Originating from typographic mistakes, sensor failures, or protocol errors. Such instances severely corrupt sample means, variances, and regression fits, and must be detected and removed (or imputed) during data cleaning.

- **Outliers as the primary analysis target:** Genuine unusual situations that constitute the central analytical goal. Paradigmatic examples include **credit card fraud detection**, **network intrusion detection**, and early diagnosis of **rare medical pathologies**.

Even when outliers represent valid measurements of rare events, it is often advisable to exclude them temporarily when computing summary metrics aimed at describing the typical population.

2.8.2 Outlier Detection Methodologies

Anomalies can be discovered across different dimensional settings:

- **Single Attribute (Univariate Analysis):**
 - *Categorical attributes:* An outlier is a category that occurs with a frequency vastly lower than all other domain classes.
 - *Numerical attributes:* Visual and analytical inspection using Box Plots (points beyond $[Q_1 - 1.5 \times \text{IQR}, Q_3 + 1.5 \times \text{IQR}]$) and standardized z -scores ($z = \frac{x-\mu}{\sigma}$).
- **Multiple Attributes (Multivariate Analysis):**
 - *2D and 3D Scatter Plots:* Reveal both global outliers (isolated with respect to the entire point cloud) and local outliers (deviating from their specific class cluster, as seen in Figure 2.6).
 - *PCA Projections:* Project high-dimensional data onto maximal-variance planes (Principal Component Analysis) to visually isolate anomalies.
 - *Clustering Algorithms:* In density-based or partition algorithms (such as DBSCAN or k-means with distance cutoffs), outliers naturally emerge as points that cannot be assigned to any major cluster or that form isolated micro-clusters.

2.8.3 Missing Values and Camouflage

The absence of values for attributes across records arises from several root causes:

- **Information not collected or withheld:** Survey respondents explicitly refuse to disclose sensitive personal data (e.g., income, weight, or age).

- **Logical non-applicability:** The attribute has no logical meaning for a subpopulation (e.g., annual employment income for school-age children, or driver's licenses for minors).
- **Sensor breakdown or network loss:** Physical hardware malfunctions, depleted sensor batteries, or packet dropouts.

Pitfall of Camouflaged Missing Values: Zeros and Default Sentinels

Frequently, a missing value is not explicitly stored as NULL or NaN, but rather recorded using conventional default or sentinel numbers (such as 0, -1, or 999). A blocked or broken sensor, for instance, often transmits a constant stream of 0s. Unless intercepted during Data Understanding, analytics software treats these sentinel numbers as true physical measurements, severely biasing means, standard deviations, correlations, and classification models.

2.8.4 Python Lab: Missing Values Detection, Sentinel Unmasking, and Imputation

Run this section in the Interactive Notebook

Notebook: `1_basics_and_understanding.ipynb` [Open in Jupyter Lab]

Local server: <http://localhost:8888> (launch command: `docker compose up jupyter`)

In the companion notebook `1_basics_and_understanding.ipynb`, missing value diagnostics, sentinel unmasking, and imputation algorithms (univariate and KNN-based) are implemented using Pandas and Scikit-learn as displayed in Listing 2.4:

Listing 2.4: Missing values detection, sentinel cleaning, and imputation strategies with Scikit-learn (SimpleImputer and KNNImputer).

```

1 import numpy as np
2 import pandas as pd
3 from sklearn.impute import SimpleImputer, KNNImputer
4
5 df = pd.read_csv('./data_notebook/data/tips.csv',
6                 index_col=0)
7
8 # 1. Inspect explicit missing values per attribute

```

```

8 print("Missing values per column:\n", df.isnull().sum())
9
10 # 2. Detect and unmask sentinel zeros (e.g., recorded
    age == 0)
11 zero_ages = (df['age'] == 0).sum()
12 print(f"Records with sentinel age == 0: {zero_ages}")
13 df['age'] = df['age'].replace(0, np.nan)
14
15 # 3. Univariate statistical imputation (median for
    numeric, mode for categorical)
16 median_imputer = SimpleImputer(strategy='median')
17 df['age_imputed'] = median_imputer.fit_transform(df[['age']])
18
19 freq_imputer = SimpleImputer(strategy='most_frequent')
20 df['smoker_imputed'] = freq_imputer.fit_transform(df[['smoker']]).ravel()
21
22 # 4. Multivariate k-Nearest Neighbors imputation
23 knn_imputer = KNNImputer(n_neighbors=3, weights='distance')
24 num_cols = ['total_bill', 'tip', 'age', 'size']
25 df[num_cols] = knn_imputer.fit_transform(df[num_cols])
26 print("Remaining missing entries after KNN imputation:",
    df[num_cols].isnull().sum().sum())

```

2.9 Operational Checklist for Data Understanding

Upon concluding the exploration stage, the analyst must have methodically executed the following operational verifications:

1. **Assess Overall Data Quality:** Audit syntactic accuracy (out-of-domain entries) and semantic consistency, check completeness, evaluate timeliness, deduplicate records, and quantify class imbalance.
2. **Detect and Analyze Outliers:** Identify univariate and multivariate outliers using visualization tools (box plots, scatter plots, PCA, parallel coordinates) and determine appropriate handling policies (noise exclusion vs target investigation).

3. **Uncover and Examine Missing Values:** Quantify null frequencies and expose camouflaged missing values hidden behind sentinel zeros or default settings.
4. **Discover and Validate Correlations and Dependencies:** Compute linear (Pearson) and monotonic rank (Spearman) correlation matrices to confirm domain hypotheses, discover unexpected linkages, and prevent multicollinearity and feature redundancy.
5. **Verify Domain-Specific Assumptions:** Test distributional premises required by downstream models or heuristics (e.g., verifying whether an attribute follows an approximately normal distribution before applying Sturges' rule or parametric tests).
6. **Compare Statistics with Expected Behavior:** Ensure that empirical means, medians, skewness, and spreads align consistently with domain expertise and business context expectations.

Chapter 3

Data Preparation and Data Cleaning

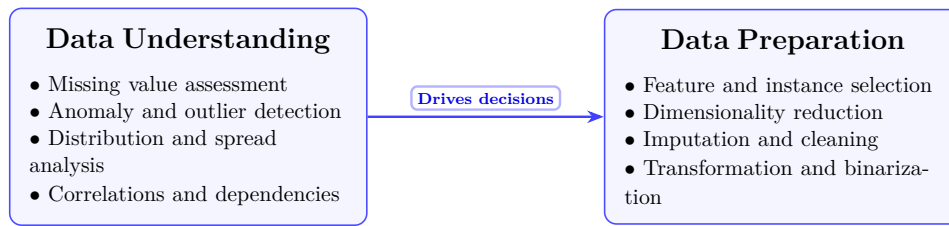
Practical Lab: Interactive Execution on Jupyter Notebook

The Python code snippets presented in this chapter correspond to the companion lab notebook:

- **Notebook:** `2_feature_engineering_....ipynb`
- **Direct access:** [[Open in Jupyter Lab](http://localhost:8888)] (<http://localhost:8888>)
- **Quickstart container:** Run from root directory: `docker compose up jupyter`
- **Local workspace folder:** `Data-Mining/notebooks/` (datasets located in `./data_notebook/`).

3.1 Objectives of Data Preparation

While *Data Understanding* (Chapter 2) investigates the structure and characteristics of the raw data, **Data Preparation** turns these findings into concrete transformations:



The primary objectives of data preparation are:

- **Improving Data Quality:** Eliminating or correcting noise, syntactic errors, and semantic inconsistencies.
- **Satisfying Algorithm Requirements:** Many data mining algorithms expect specific input structures (e.g., asymmetric binary attributes for association rules, or normalized numeric attributes for distance-based methods).
- **Computational Efficiency & Curse of Dimensionality Mitigation:** Reducing spatial and computational burdens through sampling (*data reduction*) or feature selection/projection (*dimensionality reduction*).

Core data preparation techniques include:

1. **Aggregation:** Combining attributes or records onto broader scales.
2. **Data Reduction via Sampling:** Selecting representative subsets of records.
3. **Dimensionality Reduction:** Feature selection and feature projection/extraction (PCA, SVD).
4. **Feature Creation:** Domain-driven feature engineering and mathematical construction.
5. **Discretization and Binarization:** Converting continuous attributes to ordinal categories and one-hot encoding.
6. **Attribute Transformation:** Normalization, decimal scaling, logarithmic transforms, and variance stabilization.

3.2 Data Aggregation

Definition 3.2.1 (Aggregation). **Aggregation** is the combination of two or more attributes (or objects/records) into a single attribute (or aggregated object).

Aggregation serves three essential methodological purposes:

- **Data Reduction:** Drastically reduces the number of records (rows) or attributes (columns) to process.
- **Change of Spatial or Temporal Scale:**
 - *Spatial aggregation:* Aggregating city-level measurements to regional or national levels.
 - *Temporal aggregation:* Aggregating hourly or daily observations into weekly, monthly, or yearly summaries.
- **Statistical Stability:** Aggregated data typically exhibits **less variability** than fine-grained individual observations.

3.2.1 Example: Precipitation in Australia (1982–1993)

Consider weather station rainfall records across Australia from 1982 to 1993:

- Comparing the standard deviation of average **monthly** rainfall against average **yearly** rainfall, yearly rainfall standard deviations are sharply concentrated near zero (Figure 3.1).
- Yearly aggregation compresses transient fluctuations and removes seasonal variance. However, if the mining goal is to investigate **seasonal rainfall patterns or monsoons**, the monthly histogram is substantially more informative and appropriate.

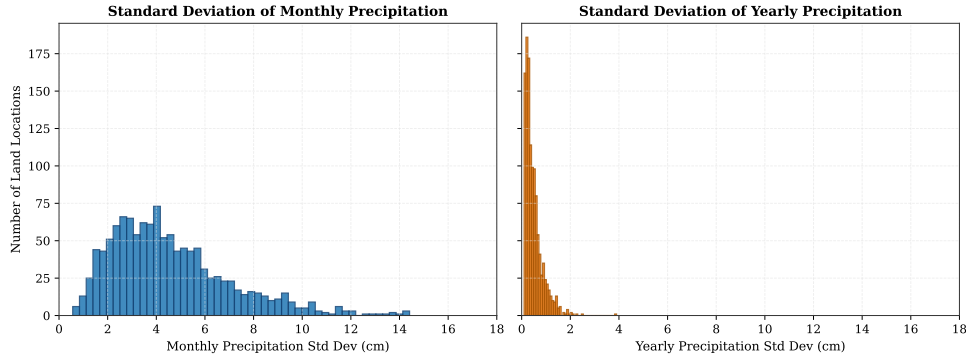


Figure 3.1: Effect of aggregation on statistical dispersion (Australia Precipitation example): on the left, standard deviation of average monthly rainfall (reveals pronounced seasonal variation); on the right, standard deviation of average yearly rainfall (demonstrates how aggregation dampens variance and stabilizes observations).

3.3 Data Reduction via Sampling

Data sampling is the primary technique used to reduce the number of records (instances).

Remark 3.3.1 (Guiding Principle of Effective Sampling). A statistical sample $\mathcal{S}$ yields analytic conclusions nearly identical to those obtained on the full population $\mathcal{D}$ if and only if $\mathcal{S}$ is **representative**. A sample is representative if it possesses approximately the same distributive properties of interest as the original dataset.

3.3.1 Sample Size Trade-offs

Sample size selection involves an analytical trade-off:

- An excessively small sample (e.g., 500 points from 8000) fragments dense clusters, degrades geometric continuity, and misses subtle local patterns.
- An appropriately sized sample (e.g., 2000 points) preserves distribution contours and topological boundaries while offering significant runtime savings.

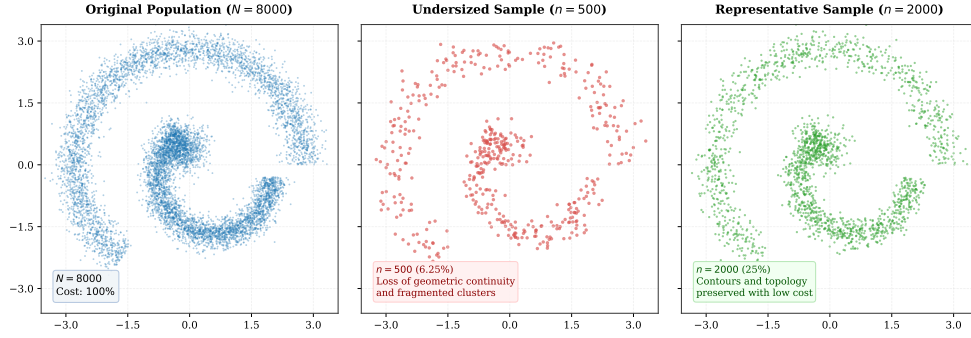


Figure 3.2: Sample size trade-off in random sampling: left panel shows the full population ($N = 8000$) with complex non-linear structures; center panel shows severe undersampling ($n = 500$, 6.25%) causing cluster fragmentation and detail loss; right panel displays an appropriately sized sample ($n = 2000$, 25%) faithfully preserving boundary contours and topologies at a substantially reduced computational footprint.

3.3.2 Types of Sampling

Two main probabilistic sampling families are distinguished:

1. **Simple Random Sampling:** Each object has an identical probability of selection:
 - **Sampling Without Replacement:** Selected items are removed from the population; each object appears at most once in the sample.
 - **Sampling With Replacement:** Selected items remain in the population and can be chosen multiple times.
2. **Stratified Sampling:** The population is partitioned into distinct subsets (*strata*), each corresponding to a specific class or density mode (Figure 3.3). Random samples are then drawn proportionally from each stratum.

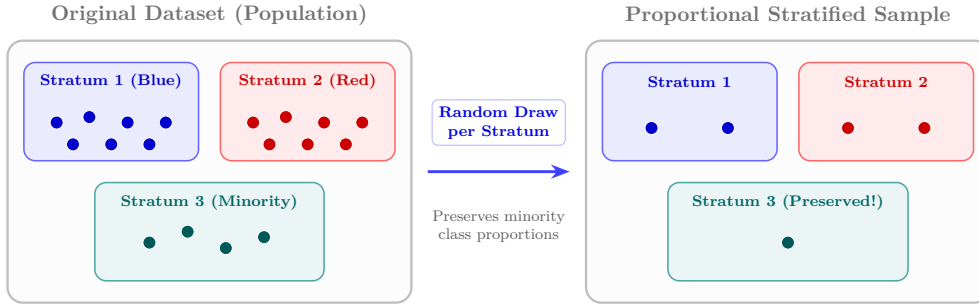


Figure 3.3: Diagram of Stratified Sampling: the population is partitioned into homogeneous strata (classes or peaked density regions); random sampling is performed independently within each stratum, safeguarding minority representations.

Stratified sampling is essential for **bimodal distributions**, **multimodal data with pronounced peaks**, and **imbalanced classes** (e.g., credit card fraud detection where fraudulent cases are $< 1\%$).

3.4 Dimensionality Reduction

In modern data mining, observations are embedded in spaces with hundreds or thousands of features:

$$\mathbf{X} \in \mathbb{R}^{n \times p} \longrightarrow \mathbf{Z} \in \mathbb{R}^{n \times k} \quad \text{con } k \ll p \quad (3.1)$$

Under this mapping:

- Objects (the n rows) retain their observational identity.
- Coordinates change: each observation is assigned k new synthetic coordinates.
- Structural preservation is method-dependent (e.g., preserving variance or class separability).

3.4.1 The Curse of Dimensionality

As the dimensionality p grows, the volume of the vector space increases exponentially relative to a finite sample size n :

- **Data Sparsity (*Sparse Neighborhoods*):** Observations become extraordinarily isolated across the volume of the unit hypercube $[0, 1]^p$. To maintain a constant sample density as dimensionality grows, the required sample size n must grow exponentially with p , an unattainable requirement in practice.
- **Distance Concentration:** The relative contrast between the maximum and minimum distance between a query and any point in the dataset asymptotically approaches zero:

$$\lim_{p \rightarrow \infty} \frac{\text{dist}_{\max} - \text{dist}_{\min}}{\text{dist}_{\min}} = 0 \quad (3.2)$$

In high-dimensional spaces, virtually all points appear approximately **equidistant** from one another.

Algorithmic Impact of the Curse of Dimensionality

The collapse of distance contrast and severe geometric sparsity directly impair the core families of data mining algorithms:

1. ***k*-Nearest Neighbors (*k*-NN) Classification:** The foundation of *k*-NN assumes that points close in feature space share the same label. When the distance from a query point to its nearest neighbor differs negligibly from that to its furthest neighbor, the very concept of a “neighbor” becomes meaningless, and predictions degrade to random noise.
2. **Centroid-Based Partitioning Clustering (*k*-Means):** Point-to-centroid assignments depend on Euclidean distance. In high dimensions, observations become nearly equidistant from all cluster centroids. Consequently, Voronoi decision boundaries become unstable and the algorithm converges to arbitrary partitions devoid of genuine semantic cohesion.
3. **Density-Based Clustering (DBSCAN):** DBSCAN discovers clusters as continuous high-density regions governed by radius ε and minimum points MinPts. In high-dimensional spaces:
 - If radius ε is set small, local hyperspheres are almost always empty, improperly classifying the vast majority of data as noise.
 - If ε is enlarged to capture points, the hypersphere incorporates distant instances across irrelevant dimensions, merging distinct natural clusters into a single massive uninformative cluster.

4. **Outlier Detection:** Distance-based and density-based anomaly detectors assume that outliers occupy isolated regions with significantly larger pairwise distances. When all pairwise distances collapse to a narrow band, distinguishing true anomalies from nominal background data becomes statistically unviable.

3.4.2 Motivations for Dimensionality Reduction

- **Visualization:** Enables visual inspection in 2 or 3 dimensions.
- **Compression:** Reduces memory footprints and algorithm execution times.
- **Noise Reduction & Multicollinearity Removal:** Discards redundant or non-informative dimensions.

Cautionary Note

Compact low-dimensional representations can obscure rare events, minor clusters, or task-relevant nuances.

3.5 Feature Subset Selection

Feature subset selection identifies the minimal set of attributes sufficient for analysis by discarding:

1. **Irrelevant Features:** Attributes containing no valuable signal for the task (e.g., student ID for GPA prediction).
2. **Redundant Features:** Attributes replicating information already encoded elsewhere (e.g., purchase price and sales tax paid).

3.5.1 Taxonomy of Feature Selection Methods

Literature classifies feature selection into three primary paradigms (Figure 3.4):

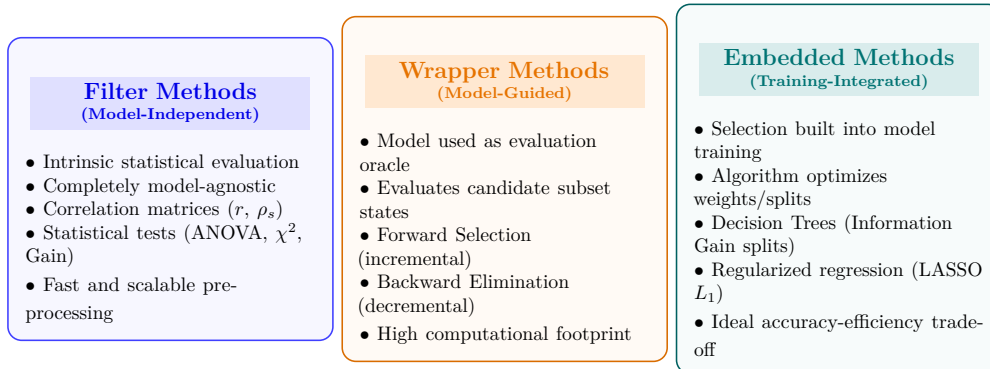


Figure 3.4: Schematic comparison between Filter, Wrapper, and Embedded feature selection paradigms.

3.5.2 Wrapper Strategies

Since an exhaustive search across all subsets of p attributes entails evaluating 2^p combinations (for $p = 20$, exceeding 1 million subsets), wrappers use greedy search heuristics:

- **Forward Selection:** Starts from an empty set $\emptyset$. In each iteration, candidates are evaluated individually and the feature granting the largest performance gain is added. The search terminates when improvement falls below a pre-set threshold.
- **Backward Elimination:** Starts with all p attributes. In each step, the attribute causing the least performance loss (or greatest gain) is pruned.

3.6 Feature Creation

Feature creation defines new synthetic attributes capturing latent dynamics more effectively than raw variables:

- **Domain-Driven Feature Construction:** Algebraic combinations reflecting domain knowledge.

Example 3.6.1 (Employee Efficiency Metric). In workplace productivity analysis, an employee is characterized by: tasks completed per

month, hours worked per month, and standard expected time per task. Instead of using raw variables in isolation, define:

$$\text{Efficiency} = \frac{\text{Hours actually spent finishing tasks}}{\text{Hours normally required to finish tasks}} \quad (3.3)$$

- **Unstructured Data (Computer Vision):** Raw pixel intensities are poorly suited for linear models. High-level features (edge descriptors, color histograms, facial landmark heatmaps) are extracted instead.
- **Feature Projection / Extraction:** Mathematical mappings from $\mathbb{R}^p$ to $\mathbb{R}^k$ ($k < p$). Linear approaches include **Principal Component Analysis (PCA)** and **Singular Value Decomposition (SVD)**; non-linear methods include *Linear Discriminant Analysis (LDA)*, *Non-Negative Matrix Factorization (NMF)*, and neural *Autoencoders*.

—

3.7 Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is the foundational unsupervised method for linear dimensionality reduction.

3.7.1 Geometric Intuition and Objectives

Real-world attributes are often linearly dependent:

- Rent expense, monthly salary, and groceries budget.
- Bank deposits, salary, and employment tenure.

PCA derives a new orthogonal coordinate system rotated relative to the original space:

1. The **first principal component (PC1)** points along the direction of maximum projected data variance.
2. The **second principal component (PC2)** is constrained to be strictly **orthogonal** to PC1 while capturing the maximum remaining variance.
3. Each successive component PC_i is mutually orthogonal to all preceding axes and maximizes residual dispersion.

3.7.2 Formal 5-Step Algorithmic Procedure

Given a data matrix $\mathbf{D} \in \mathbb{R}^{m \times n}$ with m observations and n continuous variables:

Algorithm 1: Principal Component Analysis (PCA)

Input: Raw data matrix $\mathbf{D} \in \mathbb{R}^{m \times n}$, target dimension $k \leq n$.

Output: Reduced matrix $\mathbf{Z} \in \mathbb{R}^{m \times k}$.

// Step 1: Centering and Standardization

- 1 Compute column-wise sample means $\bar{x}_j = \frac{1}{m} \sum_{i=1}^m x_{ij}$
- 2 Subtract means to produce centered matrix $\mathbf{C} \in \mathbb{R}^{m \times n}$: $c_{ij} = x_{ij} - \bar{x}_j$
- 3 (Optional/Recommended): Divide by standard deviation s_j when feature units differ

// Step 2: Covariance Matrix Computation

- 4 Compute the sample covariance matrix $\mathbf{V} \in \mathbb{R}^{n \times n}$:

$$\mathbf{V} = \frac{1}{m-1} \mathbf{C}^T \mathbf{C} \quad (3.4)$$

$\mathbf{V}$ is square, symmetric ($v_{ij} = v_{ji}$), and positive semi-definite, with diagonal elements equal to feature variances

// Step 3: Eigenvalue and Eigenvector Decomposition

- 5 Compute orthonormal eigenvectors $\mathbf{e}_j$ and eigenvalues λ_j by solving:

$$\mathbf{V} \mathbf{e}_j = \lambda_j \mathbf{e}_j \quad \text{with } \|\mathbf{e}_j\|_2 = 1 \quad (3.5)$$

// Step 4: Sorting and Selection of Top k Components

- 6 Sort eigenvalues in descending order: $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n \geq 0$
- 7 Select the top k corresponding eigenvectors $\{\mathbf{e}_1, \dots, \mathbf{e}_k\}$ to form transformation matrix $\mathbf{P} \in \mathbb{R}^{n \times k}$

// Step 5: Projection into Reduced Subspace

- 8 Project the centered matrix into the new space:

$$\mathbf{Z} = \mathbf{C} \mathbf{P} \in \mathbb{R}^{m \times k} \quad (3.6)$$

3.7.3 Significance of Covariance and Eigenvalues

- **Covariance Sign:** The sign of $\text{Cov}(x, y)$ dictates the co-variation direction:
 - $\text{Cov}(x, y) > 0$: x and y increase or decrease together.

- $\text{Cov}(x, y) < 0$: as x increases, y tends to decrease.
- $\text{Cov}(x, y) \approx 0$: no linear association.
- **Eigenvectors & Eigenvalues:** Eigenvectors define the **principal axes of maximum variance**. Each eigenvalue λ_j quantifies the variance carried by component PC_j .
- **Explained Variance Ratio:**

$$\text{Explained Variance Ratio}_j = \frac{\lambda_j}{\sum_{i=1}^n \lambda_i} \quad (3.7)$$

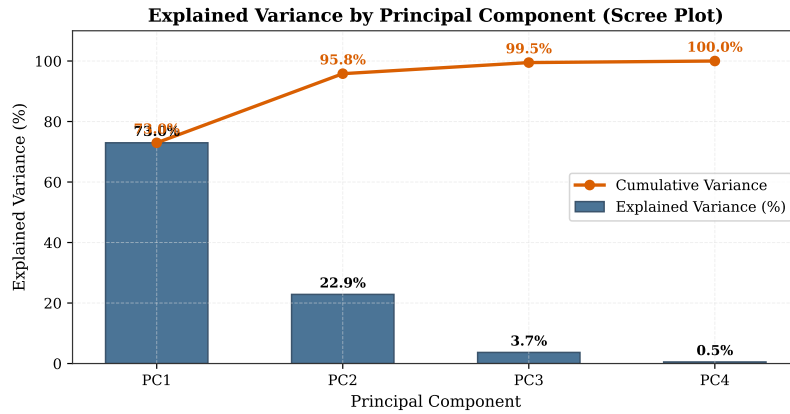


Figure 3.5: Scree plot computed on the 4 numerical attributes of the Iris dataset. The first principal component (PC1) accounts for 73.0% of total variance, while PC1 and PC2 together cover 95.8%, confirming the adequacy of a 2D projection.

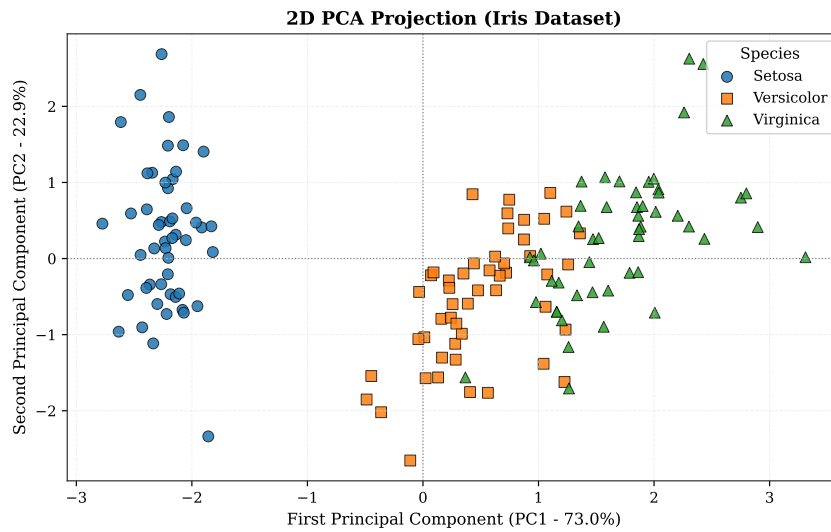


Figure 3.6: 2D PCA projection of the Iris dataset along PC1 and PC2. Unsupervised projection preserves linear separability for Setosa and clear morphological separation between Versicolor and Virginica without needing ground-truth labels.

3.7.4 Python Lab: PCA and Scree Plot

Run this section in the Interactive Notebook

Notebook: [2_feature_engineering_...ipynb](#) [Open in Jupyter Lab]

Local server: <http://localhost:8888> (launch command: `docker compose up jupyter`)

In the companion notebook `2_feature_engineering_...ipynb`, unsupervised dimensionality reduction via PCA, eigenvalue variance decomposition, and the Scree Plot are implemented using Scikit-learn as shown in Listing 3.1:

Listing 3.1: PCA execution, eigenvalue variance decomposition, and Scree Plot with Scikit-learn.

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from sklearn.decomposition import PCA
5
6 df = pd.read_csv('./data_notebook/data/correct_tips.csv',
7                 , index_col=0)
```



```

7 numeric_cols = ['total_bill', 'tip', 'size', 'age']
8 X = df[numeric_cols]
9
10 # 1. Instantiate and fit full PCA model
11 pca = PCA()
12 X_pca = pca.fit_transform(X)
13
14 # 2. Extract eigenvalues (explained variance) and
    variance ratios
15 eigenvalues = pca.explained_variance_
16 var_ratio = pca.explained_variance_ratio_
17 cum_var = np.cumsum(var_ratio)
18
19 for i, (eig, ratio, c) in enumerate(zip(eigenvalues,
    var_ratio, cum_var), 1):
20     print(f"PC{i}: Eigenvalue={eig:.3f} | Variance={
        ratio*100:.1f}% | Cumulative={c*100:.1f}%")
21
22 # 3. Scree Plot of principal component eigenvalues
23 plt.figure(figsize=(7, 4))
24 plt.plot(range(1, len(eigenvalues) + 1), eigenvalues,
    marker='o', linestyle='--', color='b')
25 plt.title("Scree Plot: Principal Component Eigenvalues")
26 plt.xlabel("Principal Component")
27 plt.ylabel("Eigenvalue (Explained Variance)")
28 plt.grid(True)
29 plt.show()

```

3.8 Missing Value and Anomaly Cleaning

3.8.1 Types and Origins of Data Anomalies

Raw datasets frequently suffer from collection imperfections caused by hardware sensor faults, survey non-responses, data transcription errors, or evolving logging formats:

- **Missing Values:** Explicit unrecorded fields represented as NULL or NaN.
- **Unknown Values:** Populated records lacking verifiable real-world semantic meaning.

- **Invalid Values:** Entries violating domain integrity constraints (e.g., negative ages, impossible exam grades, or corrupted identifier strings).

3.8.2 Methodological Strategies for Missing Data Handling

In practical data mining workflows, missing data remediation falls into three complementary approaches:

1. Elimination (*Data / Attribute Removal*):

- *Listwise / Case Deletion:* Removing entire records containing one or more null attributes. This is only admissible when missing rates are negligible ($< 3 \div 5\%$) and data is Missing Completely at Random (MCAR). Otherwise, discarding records introduces severe selection bias.
- *Feature Deletion:* Dropping entire columns. Recommended when an attribute has excessive missingness (exceeding $50 \div 60\%$) and possesses low predictive or explanatory relevance.
- *Pairwise Deletion:* Used during bivariate statistical computations (such as covariance or correlation matrices) by omitting only instances lacking both attributes, thereby preserving available sample size.

2. Value Imputation (*Value Estimation*):

- **Global Estimators (Mean, Median, Mode):** The arithmetic mean is used for symmetric continuous attributes; the median is standard for skewed distributions with heavy tails and outliers; the mode is applied to categorical features. *Drawback:* artificially dampens attribute variance and attenuates cross-feature covariances.
- **Empirical Probability Distribution Sampling:** Replacement values are drawn randomly according to the observed empirical frequency distribution of valid observations. *Benefit:* preserves original empirical dispersion, variance, and distributional shape.
- **Stratified Imputation across Segments or Clusters:** Data is grouped by target class, demographic segment, or unsupervised clusters (e.g., k-means); missing entries are replaced by the mean or median computed strictly within that subgroup (e.g., imputing

missing salary using the median salary of the subject’s exact job title).

- **Supervised Predictive Modeling:** The incomplete feature is framed as target variable Y . A predictive model (decision tree, linear regression, or k -NN) is trained on complete features to infer missing entries with maximal contextual accuracy.

3. **Direct Algorithmic Tolerance (*Ignore Missing Values*):** Several data mining algorithms naturally accommodate missing entries during training without preprocessing:

- *Decision Trees (e.g., C4.5):* During node splitting, instances with missing values are partitioned proportionally across child branches according to observed frequencies, or directed to a dedicated missing branch.
- *Association Rules:* In binary transaction matrices, an absent value intrinsically represents item non-purchase (0), seamlessly aligning with the algorithm’s operational semantics.

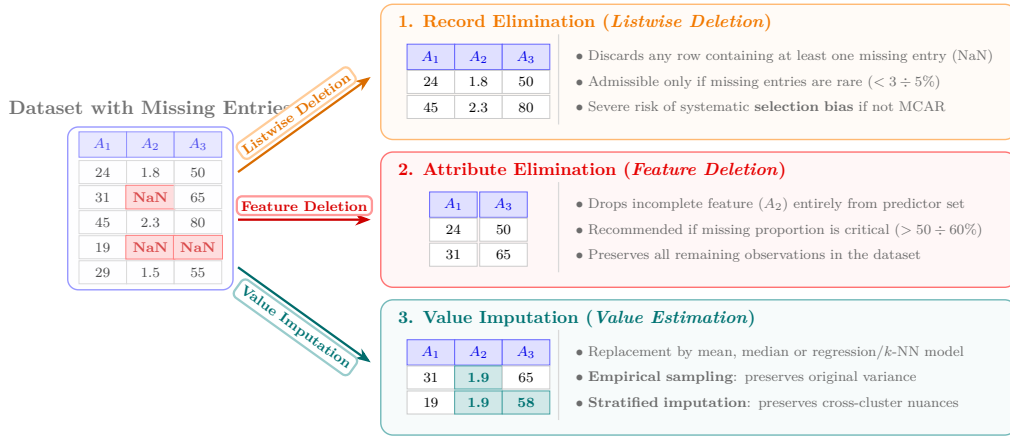


Figure 3.7: Schematic overview of missing value management strategies: record or attribute elimination versus value imputation.

3.8.3 Univariate Outlier Detection and Treatment

In preprocessing workflows, analysts deploy two formal criteria (Figure 3.8):

1. **Tukey Interquartile Range (IQR) Fences:**

$$\text{IQR} = Q_3 - Q_1, \quad L = Q_1 - 1.5 \times \text{IQR}, \quad U = Q_3 + 1.5 \times \text{IQR} \quad (3.8)$$

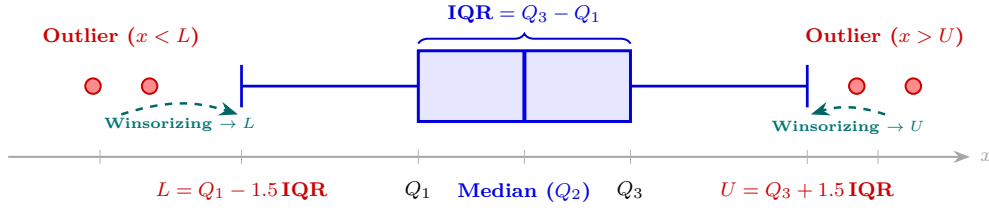
An observation x is an outlier if $x < L$ or $x > U$.

- *Winsorization*: Capping values at L or U .
- *Median Replacement*: Overwriting outliers with Q_2 .

2. **Parametric Z-Score Approach**: For normally distributed data:

- 95% of observations fall within $[-1.96, +1.96]$.
- 99% fall within $[-2.58, +2.58]$.
- Standard extreme outlier threshold: $|z| > 3.29$ (probability $< 0.1\%$).
- *Outlier treatment*: Truncating to $z = \pm 3.0$ or replacing with the mean ($z = 0$).

A) Non-Parametric Interquartile Range Rule (Tukey IQR Fences)



B) Parametric Z-Score Rule (Gaussian Distribution $\mathcal{N}(\mu, \sigma^2)$)

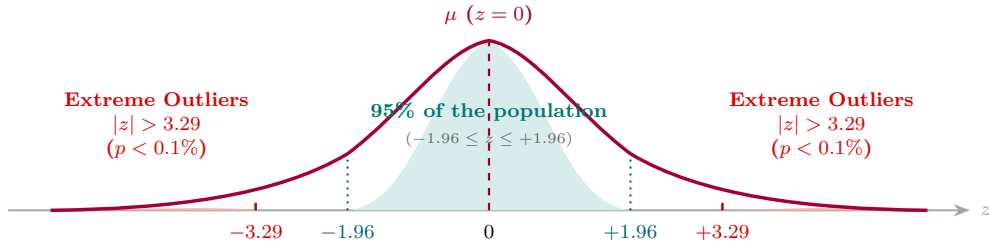


Figure 3.8: Comparison of univariate outlier detection methods: non-parametric Tukey IQR fences versus parametric Gaussian Z-Score rule.

3.9 Discretization and Binarization

3.9.1 Definition and Benefits of Discretization

Discretization converts a continuous attribute into an ordinal discrete attribute by binning infinite real values into k categorical intervals.

- **Interpretability:** Produces concise categories (such as Young, Adult, Senior).
- **Robustness:** Attenuates measurement noise and isolated outliers.
- **Algorithm Compatibility:** Required by associative rule algorithms and specific naive Bayes models.

3.9.2 Unsupervised Discretization (Binning)

1. **Equal-Width (Natural) Binning:** Partitions data range into k equal-width intervals δ :

$$\delta = \frac{x_{\max} - x_{\min}}{k} \quad (3.9)$$

Value x_j belongs to interval $i \in \{0, \dots, k-1\}$ if:

$$x_j \in [x_{\min} + i\delta, x_{\min} + (i+1)\delta) \quad (3.10)$$

Drawback: Heavily skewed data clusters into a few bins, leaving others nearly empty.

2. **Equal-Frequency (Quantile) Binning:** Partitions sorted data into k bins containing roughly equal counts $f = \frac{N}{k}$: Sorted element x_i ($i \in \{0, \dots, N-1\}$) falls into bin j if:

$$j \times f \leq i < (j+1) \times f \quad (3.11)$$

Advantage: Maximizes bin entropy and avoids empty partitions.

3.9.3 Criteria for Choosing Number of Bins

- **Sturges' Rule (1929):** Recommended for symmetric unimodal distributions:

$$k = 1 + \frac{10}{3} \log_{10}(N) \approx 1 + 3.322 \log_{10}(N) = 1 + \log_2(N) \quad (3.12)$$

- **Scott's Rule (1979):** Directly calculates optimal bin width h based on sample standard deviation s :

$$h = \frac{3.5 \times s}{\sqrt[3]{N}} \quad (3.13)$$

3.9.4 Supervised Discretization: Entropy-Based Splitting

In supervised classification, discretization maximizes **class purity** within intervals by minimizing conditional class entropy.

1. Evaluates candidate split thresholds T along the sorted range of feature X .
2. For partitions $S_1 = \{x \leq T\}$ and $S_2 = \{x > T\}$, computes weighted entropy:

$$E(X, T) = \frac{|S_1|}{|S|} \text{Entropy}(S_1) + \frac{|S_2|}{|S|} \text{Entropy}(S_2) \quad (3.14)$$

where $\text{Entropy}(S) = -\sum_{c=1}^C p_c \log_2(p_c)$.

3. Selects threshold T^* that maximizes Information Gain ($E(X) - E(X, T)$).
4. Recursively splits the interval with highest entropy until reaching a target bin count or stopping criterion.

3.9.5 Binarization

Binarization maps continuous attributes (following discretization) or categorical attributes into one or more binary variables (0 or 1). Data mining literature contrasts two primary paradigms:

1. **Compact Positional Binary Encoding (*Integer-to-Binary*)**: A categorical attribute with m distinct categories is mapped to integers $\{0, 1, \dots, m-1\}$ and expressed in base-2 positional format using $k = \lceil \log_2(m) \rceil$ bits.

- *Example*: For a feature with $m = 4$ states (e.g. Red, Blue, Green, Yellow), $\lceil \log_2(4) \rceil = 2$ bits are needed:

$$\text{Red} \rightarrow 00_2, \quad \text{Blue} \rightarrow 01_2, \quad \text{Green} \rightarrow 10_2, \quad \text{Yellow} \rightarrow 11_2 \quad (3.15)$$

- *Critical Deficiencies and Side Effects*: While space-efficient, positional encoding introduces **entirely artificial metric distances and spurious correlations**:

- *Distorted Distance*: The Hamming distance between Red (00) and Blue (01) is 1, whereas between Red (00) and Yellow (11) it is 2. Machine learning models erroneously perceive Yellow as “twice as far” from Red as Blue, violating nominal symmetry.
- *Spurious Correlations*: Accidental bit sharing (e.g., the leading bit 1 shared by Green and Yellow) creates artificial linear relationships between synthetic coordinates.

2. **Asymmetric Binarization / One-Hot Encoding**: For a feature with m categories, m distinct binary indicators $x_1, \dots, x_m$ are introduced, where $x_j = 1$ if the instance assumes modality j , and $x_j = 0$ otherwise (Table 3.1).

- *Geometric Invariance*: All categories are mutually orthogonal in $\mathbb{R}^m$ and equidistant ($\|\mathbf{x}_i - \mathbf{x}_j\|_2 = \sqrt{2}$ for $i \neq j$), preserving the unranked nature of nominal variables.
- *Core Requirement for Association Analysis (Association Rules)*: Frequent pattern mining algorithms (such as Apriori or FP-Growth) model transactions as sets of asymmetric binary items:
 - Item presence (1, e.g., “the customer bought **Bread**”) conveys active discriminative information and contributes to support and confidence.
 - Item absence (0) constitutes the passive, non-informative background state (in a supermarket catalog of 50 000 products, each basket contains very few items).
 - With symmetric binary encoding or compact positional bits, algorithms would extract billions of uninformative or misleading associations based on joint non-presence (e.g., “customers who do not buy caviar do not buy helicopters”). One-Hot encoding isolates each feature level as an independent asymmetric item.

Categorical Value	Index	x_1	x_2	x_3	x_4	x_5
awful	0	1	0	0	0	0
poor	1	0	1	0	0	0
OK	2	0	0	1	0	0
good	3	0	0	0	1	0
great	4	0	0	0	0	1

Table 3.1: One-Hot Encoding of a categorical attribute with $m = 5$ states into 5 asymmetric binary indicator attributes.

3.9.6 Python Lab: Discretization and Categorical Encoding

Run this section in the Interactive Notebook

Notebook: `2_feature_engineering....ipynb` [[Open in Jupyter Lab](#)]

Local server: <http://localhost:8888> (launch command: `docker compose up jupyter`)

In the companion notebook `2_feature_engineering....ipynb`, continuous attribute discretization and categorical encoding strategies (One-Hot Encoding and Target Encoding) are implemented using Scikit-learn as displayed in Listing 3.2:

Listing 3.2: Discretization with `KBinsDiscretizer` and Categorical Encoding (One-Hot and Target Encoding) with Scikit-learn.

```
1 import pandas as pd
2 from sklearn.preprocessing import KBinsDiscretizer,
   OneHotEncoder, TargetEncoder
3
4 df = pd.read_csv('./data_notebook/data/correct_tips.csv',
   , index_col=0)
5
6 # 1. Continuous to ordinal discretization (Equal-Width
   vs Equal-Frequency)
7 kbins_uniform = KBinsDiscretizer(n_bins=4, encode='
   ordinal', strategy='uniform')
8 df['bill_uniform'] = kbins_uniform.fit_transform(df[['
   total_bill']])
9
10 kbins_quantile = KBinsDiscretizer(n_bins=4, encode='
   ordinal', strategy='quantile')
11 df['bill_quantile'] = kbins_quantile.fit_transform(df[['
   total_bill']])
12 print("Uniform bin frequencies:\n", df['bill_uniform'].
   value_counts())
13
14 # 2. One-Hot Encoding for nominal categorical features
15 ohe = OneHotEncoder(handle_unknown='ignore',
   sparse_output=False)
16 ohe_day = ohe.fit_transform(df[['day']])
17 df_ohe = pd.DataFrame(ohe_day, columns=ohe.
```



```

18     get_feature_names_out(['day']))
19
20 # 3. Supervised Target Encoding
21 target_encoder = TargetEncoder(smooth="auto")
22 # Estimates smoothed conditional expected target value
    ('tip') per category
23 df['day_target_encoded'] = target_encoder.fit_transform(
    df[['day']], df['tip'])
24 print(df[['day', 'day_target_encoded']].drop_duplicates
    ())

```

3.10 Attribute Transformation and Variance Stabilization

3.10.1 Motivations and Formal Properties

An attribute transformation is a mapping $T : \mathcal{X} \rightarrow \mathcal{Y}$ replacing original value x with $y = T(x)$. Desirable properties include:

1. **Information Preservation:** Preserves natural monotonic ordering ($x_a < x_b \implies T(x_a) < T(x_b)$).
2. **Distribution Normalization:** Transforms skewed distributions into approximately normal, symmetric shapes.
3. **Linearization:** Straightens nonlinear bivariate relationships.
4. **Analytical Regularity:** Continuous, differentiable, and invertible mappings.

3.10.2 Exponential and Logarithmic Transformation Families

The general parametric transformation for $x > 0$ is defined as:

$$T_p(x) = \begin{cases} ax^p + b & \text{if } p \neq 0 \\ c \log(x) + d & \text{if } p = 0 \end{cases} \quad (3.16)$$

Depending on parameter p :

- **Linear Transformation** ($p = 1$): Preserves skewness and correlation exactly; used for unit scaling:
 - Currency conversion: Lira = $1936.27 \times$ Euro, with $p = 1, a = 1936.27, b = 0$.
 - Temperature conversion: $^{\circ}\text{C} = \frac{5}{9}(^{\circ}\text{F} - 32)$ ($p = 1, a = 5/9, b = -160/9$).
- **Logarithmic Transformation** ($p = 0$): Applicable to positive values $x > 0$ under the general specification $T(x) = c \log(x) + d$. Compresses long right tails in log-normal distributions, stabilizing variance and mitigating extreme outliers.

Remark 3.10.1 (Logarithms and Seasonality). Logarithmic transformations do not eliminate seasonal cycles, but compress amplitude fluctuations that scale with the series level.

Example 3.10.1 (Logarithmic Transformation on Sparse Data: Bar and Beer Example). Consider the lecture case study (Table 3.2), analyzing sales earnings (*Gain*) across beer brands in different commercial bars.

Bar	Beer Brand	Original Gain	Transformed Gain	$\log_{10}(\text{Gain})$
Bar A	Bud	20		1.3010
Bar A	Becks	10000		4.0000
Bar C	Bud	300		2.4771
Bar D	Bud	400		2.6021
Bar D	Becks	5		0.6990
Bar E	Becks	120		2.0792
Bar E	Bud	120		2.0792
Bar F	Bud	11000		4.0414
Bar G	Bud	1300		3.1139
Bar H	Bud	3200		3.5051
Bar H	Becks	1000		3.0000
Bar I	Bud	135		2.1303

Table 3.2: Effect of logarithmic transformation ($\log_{10}$) on beer sales across bars: compression of severe dispersion.

Statistical Diagnosis and Compression of Sparse Data:

- *Raw Series*: Sample mean $\bar{x} \approx 2481.82$, Standard deviation $s \approx 4079.02$, Minimum 5, 1st Quartile 120, Median 400, 3rd Quartile

2250, Maximum 11000. The standard deviation substantially exceeds the mean ($s > \bar{x}$) and the maximum spans over three orders of magnitude above the minimum ($2200\times$). Data is heavily skewed and sparse in the upper range (“*Data are sparse!*”), severely distorting distance metrics and regression loss functions.

- *Transformed Series*: Applying $y = \log_{10}(\text{Gain})$ maps the entire dynamic range into the interval $[0.6990, 4.0414]$. Multiplicative scales (orders of magnitude) become uniform linear intervals, stabilizing variance and restoring distributional balance (Figure 3.9).

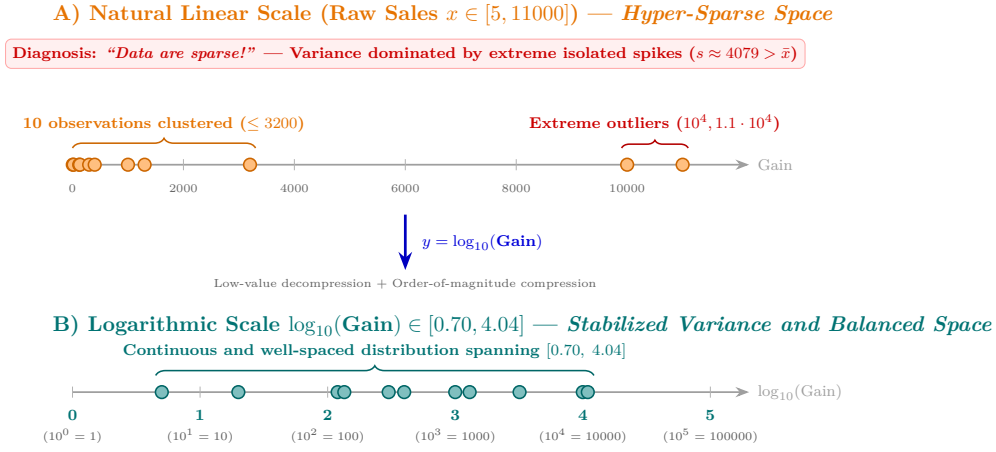


Figure 3.9: Compression and regularization effect of logarithmic transformation on beer sales data: transition from a skewed, outlier-dominated space to a homogeneous distribution with stabilized variance.

- **Square Root Transformation** ($p = 1/2$): Stabilizes variance in Poisson count processes where variance equals the mean.
- **Reciprocal Transformation** ($p < 0$, typically $p = -1$): Suited for time series where variance expands super-linearly relative to the mean.

3.10.3 Normalization Methods

Normalization unifies variable ranges, preventing high-magnitude features from dominating Euclidean distance calculations:

1. **Min-Max Normalization** (Scaling to interval $[v'_{\min}, v'_{\max}]$):

$$v' = \frac{v - \min_A}{\max_A - \min_A} \times (v'_{\max} - v'_{\min}) + v'_{\min} \quad (3.17)$$

Standard unit-interval scaling $[0, 1]$:

$$v' = \frac{v - \min_A}{\max_A - \min_A} \quad (3.18)$$

2. Z-Score Standardization (Zero Mean, Unit Variance):

$$v' = \frac{v - \bar{x}_A}{s_A} \quad (3.19)$$

where $\bar{x}_A$ is the sample mean and s_A is the sample standard deviation.

3. Decimal Scaling Normalization: Divides by powers of 10 so that the maximum absolute value falls below 1:

$$v' = \frac{v}{10^j} \quad (3.20)$$

where j is the smallest positive integer such that $\max(|v'|) < 1$.

Example 3.10.2 (Decimal Scaling). Consider the following sequence of values:

$$V = \{-10, 201, 301, -401, 501, 601, 701\}$$

- Maximum absolute value: $m = \max(|v|) = 701$.
- Since $701 < 1000 = 10^3$, the minimal integer is $j = 3$.
- Dividing each element by 1000 yields:

$$V' = \{-0.01, 0.201, 0.301, -0.401, 0.501, 0.601, 0.701\} \quad (3.21)$$

3.10.4 Python Lab: Standardization vs Min-Max Scaling and Inverse Transformation

Run this section in the Interactive Notebook

Notebook: `2_feature_engineering_...ipynb` [Open in Jupyter Lab]

Local server: `http://localhost:8888` (launch command: `docker compose up jupyter`)

In the companion notebook `2_feature_engineering_...ipynb`, practical comparison between `StandardScaler` and `MinMaxScaler`, alongside restoring original physical scales using `inverse_transform`, is implemented as shown in Listing 3.3:

Listing 3.3: Standardization with StandardScaler, Min-Max scaling, and reconstruction with inverse_transform.

```
1 import pandas as pd
2 from sklearn.preprocessing import StandardScaler,
  MinMaxScaler
3
4 df = pd.read_csv('./data_notebook/data/correct_tips.csv',
  , index_col=0)
5 num_cols = ['total_bill', 'tip', 'age']
6
7 # 1. Z-Score Standardization (mean = 0, std = 1)
8 scaler_std = StandardScaler()
9 df_standardized = df.copy()
10 df_standardized[num_cols] = scaler_std.fit_transform(df[
  num_cols])
11
12 # 2. Min-Max Normalization (bounded interval [0, 1])
13 scaler_minmax = MinMaxScaler()
14 df_normalized = df.copy()
15 df_normalized[num_cols] = scaler_minmax.fit_transform(df
  [num_cols])
16
17 # 3. Inverse transformation to restore original scale
18 df_reconstructed = df_standardized.copy()
19 df_reconstructed[num_cols] = scaler_std.
  inverse_transform(df_standardized[num_cols]).round(2)
20 print("Reconstructed features via inverse_transform:\n",
  df_reconstructed[num_cols].head(3))
```

3.10.5 Python Lab: Non-Linear Dimensionality Reduction with t-SNE vs PCA

Run this section in the Interactive Notebook

Notebook: [2_feature_engineering_....ipynb](#) [Open in Jupyter Lab]

Local server: <http://localhost:8888> (launch command: docker compose up jupyter)

In the companion notebook `2_feature_engineering_....ipynb`, non-linear neighbor embedding with *t*-SNE and comparative 2D visualization against linear PCA are structured as shown in Listing 3.4:

Listing 3.4: Non-linear manifold learning with t-SNE and 2D visual comparison against PCA in Scikit-learn.

```
1 import pandas as pd
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4 from sklearn.decomposition import PCA
5 from sklearn.manifold import TSNE
6
7 df = pd.read_csv('./data_notebook/data/correct_tips.csv',
8                 , index_col=0)
9 num_cols = ['total_bill', 'tip', 'age', 'size']
10 X = df[num_cols]
11
12 # 1. Global linear projection via PCA (2 components)
13 X_pca = PCA(n_components=2).fit_transform(X)
14 df['pca_1'], df['pca_2'] = X_pca[:, 0], X_pca[:, 1]
15
16 # 2. Non-linear neighbor embedding via t-SNE
17 tsne = TSNE(n_components=2, perplexity=25, random_state
18             =42, init='pca', learning_rate='auto')
19 X_tsne = tsne.fit_transform(X)
20 df['tsne_1'], df['tsne_2'] = X_tsne[:, 0], X_tsne[:, 1]
21
22 # 3. Side-by-side 2D representation comparison
23 fig, axes = plt.subplots(1, 2, figsize=(13, 5))
24 sns.scatterplot(data=df, x='pca_1', y='pca_2', hue='time',
25                , style='sex', ax=axes[0])
26 axes[0].set_title("Global Linear Representation: PCA (2D)")
27
28 sns.scatterplot(data=df, x='tsne_1', y='tsne_2', hue='time',
29                , style='sex', ax=axes[1])
30 axes[1].set_title("Non-linear Neighborhood Embedding: t-SNE (2D)")
31 plt.tight_layout()
32 plt.show()
```

Chapter 4

Data Similarity and Introduction to Cluster Analysis

4.1 Proximity: Similarity and Dissimilarity

Every Data Mining technique that involves comparing objects — from classification to clustering, from information retrieval to anomaly detection — relies on a fundamental concept: **proximity**, an umbrella term encompassing both *similarity* and *dissimilarity*.

Definition 4.1.1 (Similarity). **Similarity** is a numerical measure of how alike two data objects are. It is higher when objects are more alike, and typically falls in the range $[0, 1]$.

Definition 4.1.2 (Dissimilarity). **Dissimilarity** is a numerical measure of how different two data objects are. It assumes its minimum value (often 0) when objects are identical; the upper limit varies depending on the chosen measure.

The relationship between the two is intuitive: low dissimilarity corresponds to high similarity. In practice, transformations such as $s = 1 - d$ or $s = e^{-d}$ convert distance into similarity.

4.1.1 Proximity for a Single Attribute

Before tackling the multi-dimensional case, it is useful to examine how proximity is defined for a single attribute depending on its type:

Attribute Type	Dissimilarity	Similarity
Nominal	$d = 0$ if $p = q$; $d = 1$ otherwise	$s = 1$ if $p = q$; $s = 0$ otherwise
Ordinal	$d = \frac{ p - q }{n - 1}$ (values mapped to $0, \dots, n - 1$)	$s = 1 - d$
Interval / Ratio	$d = p - q $	$s = -d, \quad s = \frac{1}{1 + d},$ or $s = e^{-\alpha d}$

Table 4.1: Proximity measures for a single attribute by data type.

4.2 Euclidean Distance and Metric Properties

When data objects are points in $\mathbb{R}^n$, the most natural distance measure is the **Euclidean distance**.

Definition 4.2.1 (Euclidean Distance). For two points $\mathbf{x} = (x_1, \dots, x_n)$ and $\mathbf{y} = (y_1, \dots, y_n)$ in $\mathbb{R}^n$, the **Euclidean distance** (L_2 norm) is:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{k=1}^n (x_k - y_k)^2} \quad (4.1)$$

Standardization Required

If attribute scales differ (e.g., one attribute in $[0, 1]$ and another in $[0, 10,000]$), attributes with a larger range will dominate the distance computation. **Standardization** (subtract mean, divide by standard deviation) should be applied before computing Euclidean distances.

Example 4.2.1 (Euclidean Distance Matrix). Consider four points in the plane:

Point	x	y
p_1	0	2
p_2	2	0
p_3	3	1
p_4	5	1

The resulting Euclidean distance matrix is:

	p_1	p_2	p_3	p_4
p_1	0	2.828	3.162	5.099
p_2	2.828	0	1.414	3.162
p_3	3.162	1.414	0	2.000
p_4	5.099	3.162	2.000	0

4.2.1 Properties of a Metric

A distance function $d(\mathbf{x}, \mathbf{y})$ satisfying the following three properties is called a **metric**:

1. **Positive definiteness:** $d(\mathbf{x}, \mathbf{y}) \geq 0$ for all $\mathbf{x}, \mathbf{y}$, and $d(\mathbf{x}, \mathbf{y}) = 0$ if and only if $\mathbf{x} = \mathbf{y}$.
2. **Symmetry:** $d(\mathbf{x}, \mathbf{y}) = d(\mathbf{y}, \mathbf{x})$ for all $\mathbf{x}, \mathbf{y}$.
3. **Triangle inequality:** $d(\mathbf{x}, \mathbf{z}) \leq d(\mathbf{x}, \mathbf{y}) + d(\mathbf{y}, \mathbf{z})$ for all $\mathbf{x}, \mathbf{y}, \mathbf{z}$.

Similarly, a similarity function $s(\mathbf{x}, \mathbf{y})$ satisfies:

1. $s(\mathbf{x}, \mathbf{y}) = 1$ (maximum similarity) if and only if $\mathbf{x} = \mathbf{y}$.
2. $s(\mathbf{x}, \mathbf{y}) = s(\mathbf{y}, \mathbf{x})$ for all $\mathbf{x}, \mathbf{y}$ (symmetry).

4.3 Minkowski Distance: A Family of Metrics

The Euclidean distance is a special case of a more general family, the **Minkowski distance**.

Definition 4.3.1 (Minkowski Distance). For two points $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ and parameter $r \geq 1$, the **Minkowski distance** (L_r norm) is:

$$d(\mathbf{x}, \mathbf{y}) = \left(\sum_{k=1}^n |x_k - y_k|^r \right)^{1/r} \quad (4.2)$$

The parameter r controls the “shape” of the unit ball associated with the norm:

Example 4.3.1 (Comparison of L_1 , L_2 , and L_∞ distances). Using the same four points from the previous example:

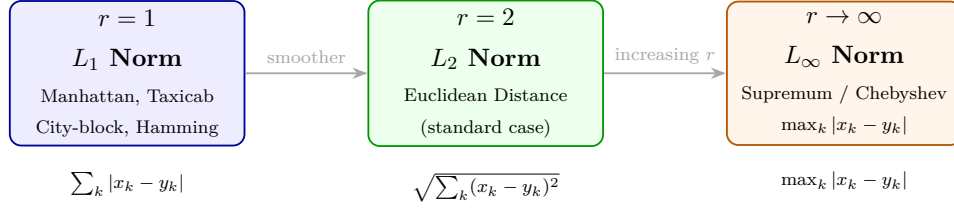


Figure 4.1: The Minkowski family of norms as the parameter r varies.

L_1	p_1	p_2	p_3	p_4
p_1	0	4	4	6
p_2	4	0	2	4
p_3	4	2	0	2
p_4	6	4	2	0

L_∞	p_1	p_2	p_3	p_4
p_1	0	2	3	5
p_2	2	0	1	3
p_3	3	1	0	2
p_4	5	3	2	0

Under L_1 : $d(p_1, p_2) = |0 - 2| + |2 - 0| = 4$. Under L_∞ : $d(p_1, p_2) = \max(|0 - 2|, |2 - 0|) = 2$.

The L_1 norm is especially useful for high-dimensional sparse data. The **Hamming distance** — the number of differing bits between two binary vectors — is a special case of the L_1 norm.

4.4 Similarity Between Binary Data

When attributes are **binary** (present/absent, true/false), Euclidean distance loses semantic meaning. Instead, we reason in terms of *matches* and *mismatches*.

Given two objects p and q with binary attributes, we define the following **contingency table**:

	$q = 0$	$q = 1$
$p = 0$	M_{00} both 0	M_{01} $p=0, q=1$
$p = 1$	M_{10} $p=1, q=0$	M_{11} both 1

Definition 4.4.1 (Simple Matching Coefficient — SMC).

$$\text{SMC}(p, q) = \frac{M_{11} + M_{00}}{M_{01} + M_{10} + M_{11} + M_{00}} \quad (4.3)$$

Counts both 1-1 matches and 0-0 matches.

Definition 4.4.2 (Jaccard Coefficient).

$$J(p, q) = \frac{M_{11}}{M_{01} + M_{10} + M_{11}} \quad (4.4)$$

Ignores 0-0 matches: suitable when attribute absence is not informative (e.g., market-basket data, disease occurrence).

Example 4.4.1 (SMC vs. Jaccard). Let:

$$p = [1, 0, 0, 0, 0, 0, 0, 0, 0, 0], \quad q = [0, 0, 0, 0, 0, 0, 1, 0, 0, 1]$$

- $M_{11} = 0, M_{10} = 1, M_{01} = 2, M_{00} = 7$

$$\text{SMC} = \frac{0 + 7}{2 + 1 + 0 + 7} = 0.7, \quad J = \frac{0}{2 + 1 + 0} = 0$$

SMC is high because of the 7 shared zeros. Jaccard is 0 because there are no co-occurrences of 1. For market-basket data, Jaccard is the appropriate choice.

4.5 Cosine Similarity

In document data (word frequency or TF-IDF vectors), vectors are typically high-dimensional and sparse. Euclidean distance is sensitive to document length; **cosine similarity** normalizes by vector magnitude and measures only the angle between vectors.

Definition 4.5.1 (Cosine Similarity). For two vectors $\mathbf{d}_1, \mathbf{d}_2 \in \mathbb{R}^n$:

$$\cos(\mathbf{d}_1, \mathbf{d}_2) = \frac{\mathbf{d}_1 \cdot \mathbf{d}_2}{\|\mathbf{d}_1\| \|\mathbf{d}_2\|} \quad (4.5)$$

where $\mathbf{d}_1 \cdot \mathbf{d}_2 = \sum_k d_{1k} d_{2k}$ and $\|\mathbf{d}\| = \sqrt{\sum_k d_k^2}$.

The value lies in $[-1, 1]$; for non-negative vectors (such as word frequencies), the range is $[0, 1]$: 1 means identical directions, 0 means orthogonal vectors.

Example 4.5.1 (Cosine Similarity on Documents).

$$\mathbf{d}_1 = [3, 2, 0, 5, 0, 0, 0, 2, 0, 0]$$

$$\mathbf{d}_2 = [1, 0, 0, 0, 0, 0, 0, 1, 0, 2]$$

$$\mathbf{d}_1 \cdot \mathbf{d}_2 = 3 \cdot 1 + 2 \cdot 1 = 5$$

$$\|\mathbf{d}_1\| = \sqrt{9 + 4 + 25 + 4} = \sqrt{42} \approx 6.481, \quad \|\mathbf{d}_2\| = \sqrt{1 + 1 + 4} = \sqrt{6} \approx 2.449$$

$$\cos(\mathbf{d}_1, \mathbf{d}_2) \approx 0.315$$

The documents share about 31.5% of “direction”, regardless of their different lengths.

4.5.1 Weighted Similarity

When attributes have different importance, a **weighted similarity** can be defined with non-negative weights w_k summing to 1:

$$s_w(\mathbf{x}, \mathbf{y}) = \frac{\sum_{k=1}^n w_k \cdot d_k \cdot s_k(x_k, y_k)}{\sum_{k=1}^n w_k \cdot d_k} \quad (4.6)$$

4.6 Correlation

Pearson correlation measures the *linear* relationship between two variables (or between two objects viewed as attribute vectors). It is computed by standardizing the vectors and taking their dot product:

$$\text{corr}(p, q) = \frac{\sum_{k=1}^n (p_k - \bar{p})(q_k - \bar{q})}{(n-1) \sigma_p \sigma_q} \quad (4.7)$$

The result lies in $[-1, 1]$: +1 perfect positive correlation, 0 no linear relationship, -1 perfect negative correlation.

4.7 Information-Based Measures: Entropy and Mutual Information

Information-theoretic measures allow quantifying proximity between categorical or continuous variables in a non-linear way, going beyond simple value-by-value comparisons.

4.7.1 Entropy

The information content of an event is inversely proportional to its probability: a certain event carries no information; a rare event carries much.

Definition 4.7.1 (Shannon Entropy). For a discrete variable X with n possible values $x_1, \dots, x_n$ and probabilities $p_1, \dots, p_n$:

$$H(X) = - \sum_{i=1}^n p_i \log_2 p_i \quad (4.8)$$

Entropy is measured in **bits** and lies in $[0, \log_2 n]$.

Boundary cases:

- If one probability equals 1 (certain event): $H = 0$ — maximum certainty, zero information.
- If all values are equally likely ($p_i = 1/n$): $H = \log_2 n$ — maximum uncertainty.

Example 4.7.1 (Fair vs. Biased Coin). For a coin with probability p of heads and $q = 1 - p$ of tails:

$$H = -p \log_2 p - q \log_2 q$$

With $p = 0.5$ (fair coin): $H = 1$ bit. With $p = 1$ (biased): $H = 0$ bits.

4.7.2 Mutual Information

Mutual Information (MI) quantifies how much information variable X provides about variable Y :

Definition 4.7.2 (Mutual Information).

$$I(X; Y) = H(X) + H(Y) - H(X, Y) \quad (4.9)$$

where $H(X, Y) = -\sum_i \sum_j p_{ij} \log_2 p_{ij}$ is the joint entropy.

$I(X; Y) = 0$ if and only if X and Y are independent. Unlike Pearson correlation, MI captures *non-linear* dependencies.

Example 4.7.2 (Mutual Information: Student Status and Grade). For a class of 100 students with variables *Student Status* (Undergraduate/Graduate) and *Grade* (A/B/C):

Grade	Status		
	Undergrad	Grad	Total
A	5	30	35
B	30	20	50
C	10	5	15
Total	45	55	100

$H(\text{Status}) \approx 0.993$ bit, $H(\text{Grade}) \approx 1.441$ bit, $H(\text{Status}, \text{Grade}) \approx 2.271$ bit

$$I(\text{Status}; \text{Grade}) = 0.993 + 1.441 - 2.271 \approx \mathbf{0.163} \text{ bit}$$

4.8 What is Cluster Analysis?

Definition 4.8.1 (Cluster Analysis). **Cluster analysis** (or **clustering**) is the process of grouping a set of data objects into **clusters** such that:

- objects *within* the same cluster are as similar as possible (*high intra-cluster cohesion*);
- objects in *different* clusters are as dissimilar as possible (*high inter-cluster separation*).

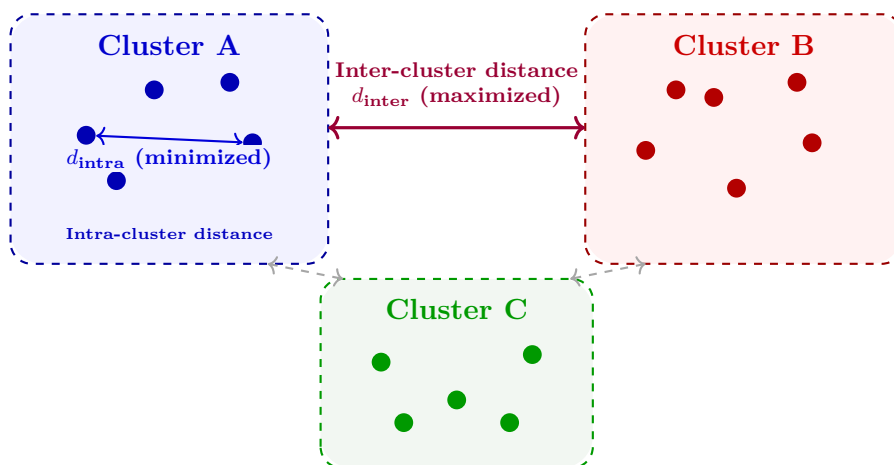


Figure 4.2: Fundamental objective of clustering: maximize inter-cluster distances (separation) and minimize intra-cluster distances (cohesion).

Clustering is an **unsupervised learning** task: unlike classification, no predefined labels are available. The algorithm must autonomously discover latent structure in the data.

4.8.1 Applications of Cluster Analysis

Clustering is applied across a wide range of domains:

- **Understanding:** group related documents by topic (IR), group genes with similar functionality (bioinformatics), group stocks with correlated price movements (finance).
- **Summarization and compression:** reduce large datasets by replacing each cluster with a representative (centroid or medoid).

- **Anomaly detection:** objects that do not belong to any cluster (or belong to very small clusters) are potential anomalies.
- **Spatial clustering:** discover geographic zones with similar environmental characteristics (e.g., precipitation patterns).

4.8.2 What is NOT Cluster Analysis?

- **Simple segmentation:** dividing students into groups alphabetically is a partition, not clustering — groups are defined by an external rule, not by data similarity.
- **Query results:** retrieving all documents containing “machine learning” produces a set, not a clustering.
- **Supervised classification:** class labels are given a priori; clustering must discover them.
- **Association Analysis:** finds local relationships between items; clustering seeks global grouping structures.

4.9 Ambiguity and Subjectivity of Clusters

No Single Correct Answer

The same point cloud can be validly interpreted as 2, 4, or 6 clusters depending on the scale of analysis, the chosen proximity measure, and the algorithm used. There is no “objectively correct” answer: clustering is always relative to the model and context.

4.10 Taxonomy of Clusterings

A **clustering** is a collection of clusters that partitions or organizes the objects. Several orthogonal taxonomies exist.

4.10.1 Partitional vs. Hierarchical Clustering

Definition 4.10.1 (Partitional Clustering). A **partitional clustering** divides objects into k disjoint subsets (clusters) such that each object belongs to exactly one cluster.

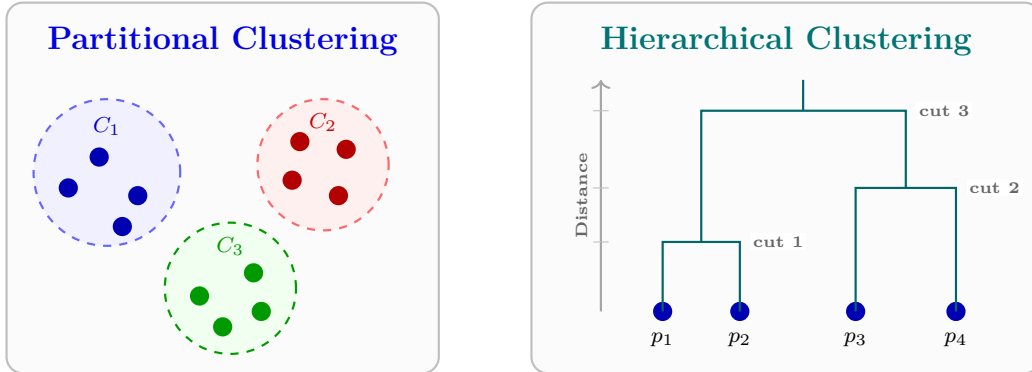


Figure 4.3: Comparison of partitional clustering (non-overlapping disjoint partitions) and hierarchical clustering (nested dendrogram with aggregation levels).

Definition 4.10.2 (Hierarchical Clustering). A **hierarchical clustering** produces a set of nested clusters organized as a tree (*dendrogram*). Each level of the tree corresponds to a different granularity of grouping.

4.10.2 Further Distinctions

- **Exclusive vs. Non-exclusive:** in non-exclusive (soft or overlapping) clustering, a point may belong to multiple clusters simultaneously (e.g., a border point).
- **Fuzzy vs. Non-fuzzy:** in fuzzy clustering, each point belongs to every cluster with a weight $w_{ik} \in [0, 1]$ such that $\sum_k w_{ik} = 1$.
- **Partial vs. Complete:** sometimes only a subset of the data is to be clustered (e.g., excluding noise).
- **Homogeneous vs. Heterogeneous:** clusters of very different sizes, shapes, and densities.

4.11 Types of Clusters

The term “cluster” can carry very different meanings depending on the adopted model.

4.11.1 Well-Separated Clusters

Definition 4.11.1 (Well-Separated Cluster). A well-separated cluster is a set of points such that any point is closer (or more similar) to every other point in its cluster than to any point in a different cluster.

This definition works well when clusters are compact and distant, but fails with elongated, irregular shapes or noise.

4.11.2 Center-Based Clusters

Definition 4.11.2 (Center-Based Cluster). A center-based cluster is a set of objects such that each object is closer to the **center** of its own cluster than to the center of any other cluster. The center can be:

- the **centroid**: the mean vector of all points in the cluster;
- the **medoid**: the most representative actual data point (minimizes mean distance to others).

This is the definition underlying K-means and K-medoids algorithms.

4.11.3 Contiguity-Based Clusters

Definition 4.11.3 (Contiguous Cluster). A contiguous cluster (transitive nearest-neighbor) is a set of points such that each point is closer to at least one other point in its cluster than to any point in another cluster.

This graph-based approach is sensitive to **noise**: a single intermediate point (a “bridge”) between two distinct clusters may erroneously merge them.

4.11.4 Density-Based Clusters

Definition 4.11.4 (Density-Based Cluster). A density-based cluster is a dense region of points separated from other dense regions by low-density regions.

This definition enables discovery of arbitrarily shaped clusters and naturally handles noise and outliers (which remain isolated in low-density regions). DBSCAN is based on this definition.

4.11.5 Clusters Defined by an Objective Function

A cluster can be defined by an **objective function** f to be minimized or maximized. The problem is generally **NP-hard**. In practice:

- **Hierarchical** algorithms typically optimize *local objectives* (greedy merges or splits).
- **Partitional** algorithms (like K-means) typically optimize *global objectives* (e.g., sum of squared errors).

Type	Membership criterion	Typical algorithms
Well-Separated	Closer to all points in own cluster	— (ideal definition)
Center-Based	Closer to own cluster's center	K-means, K-medoids
Contiguity-Based	Closer to at least one point in own cluster	Single-link HAC
Density-Based	In a high-density region	DBSCAN, OPTICS
Objective Function	Minimizes/maximizes a global function	K-means (SSE), spectral

Table 4.2: Comparison of the main cluster types.

4.12 Input Data Characteristics and Clustering Algorithms

The choice of proximity measure and clustering algorithm depends strongly on data characteristics:

- **Dimensionality:** at high dimensionality, Euclidean distance becomes less discriminative (*curse of dimensionality*) — the difference between nearest and farthest neighbors tends to vanish.
- **Sparseness:** high-dimensional sparse data (text, transactions) require Jaccard or cosine similarity.
- **Attribute type:** the proximity measure must match the attribute type (nominal, ordinal, continuous, binary).

- **Special relationships:** spatial or temporal autocorrelation in data must be accounted for.
- **Noise and outliers:** interfere with most algorithms; DBSCAN handles them natively, while K-means is very sensitive.

Main Clustering Algorithm Families

The three major algorithmic paradigms for clustering are:

1. **K-means and variants:** partitional, optimizes sum of squared errors (SSE), sensitive to outliers and the choice of k .
2. **Hierarchical clustering:** produces a dendrogram; agglomerative (*bottom-up*) or divisive (*top-down*); does not require k in advance.
3. **Density-based clustering (DBSCAN):** discovers arbitrarily shaped clusters, robust to noise, requires density parameters (ϵ , *MinPts*) instead of k .

Each family will be covered in detail in subsequent chapters.

Bibliography

- [1] Usama Fayyad, Gregory Piatetsky-Shapiro, and Padhraic Smyth. From data mining to knowledge discovery in databases. *AI Magazine*, 17(3):37–54, 1996.